DAM

SISTEMAS INFORMÁTICOS

16/09/2026

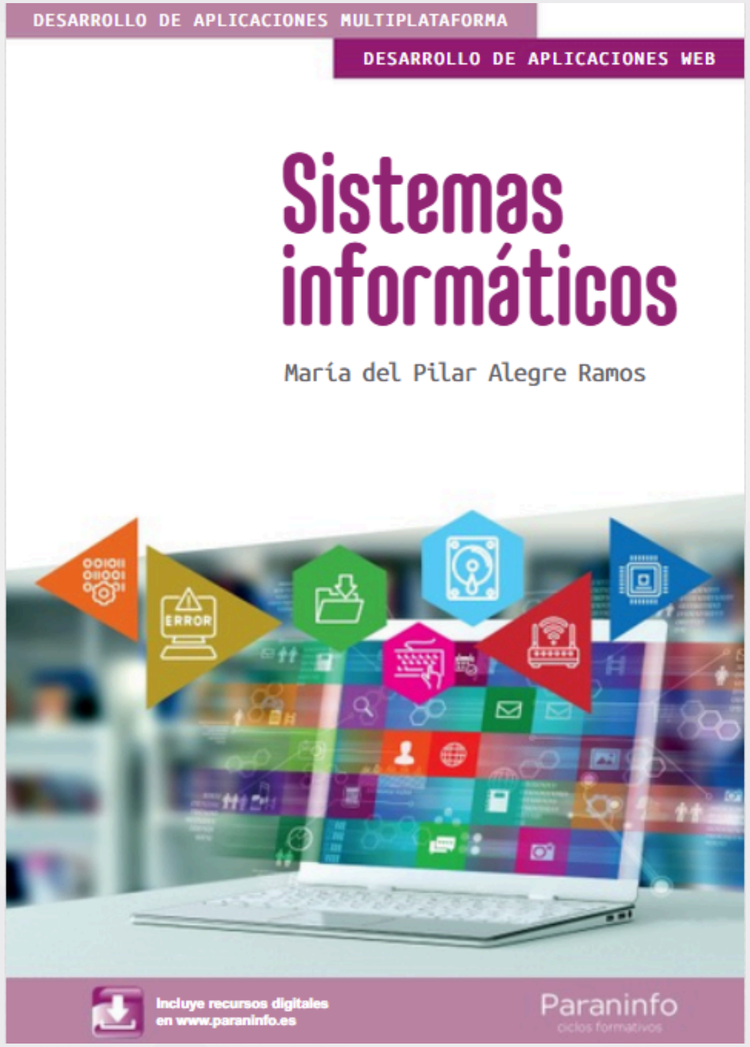
Profesora:
Begoña Rodríguez Ferreras

begona.rodfer.1@educa.jcyl.es

RESULTADOS DE APRENDIZAJE

RA1. Evalúa **sistemas informáticos**, identificando sus componentes y características.
RA2. Instala **sistemas operativos** planificando el proceso e interpretando documentación técnica.
RA3. Gestiona la información del sistema identificando las estructuras de almacenamiento y aplicando medidas para asegurar la integridad de los datos.
RA4. Gestiona sistemas operativos **utilizando comandos y herramientas gráficas** y evaluando las necesidades del sistema.
RA5. Interconecta **sistemas en red** configurando dispositivos y protocolos.
RA6. Opera **sistemas en red** gestionando sus recursos e identificando las restricciones de seguridad existentes.
RA7. Elabora **documentación** valorando y **utilizando aplicaciones informáticas de propósito general**.

LIBRO


DESARROLLO DE APLICACIONES MULTIPLATAFORMA
DESARROLLO DE APLICACIONES WEB
Sistemas informáticos
María del Pilar Alegre Ramos
ERROR
Incluye recursos digitales en www.paraninfo.es
Paraninfo
ciclos formativos

RA1

a) Se han reconocido los **componentes físicos** de un **sistema informático** y sus **mecanismos de interconexión**.

b) Se han **clasificado los tipos de memorias**, señalando sus características e identificando sus prestaciones y la función que desarrollan en el conjunto del sistema.

c) Se ha verificado el proceso de puesta en marcha de un equipo.

d) Se han clasificado, instalado y configurado diferentes tipos de **dispositivos periféricos**.

e) Se han identificado los **tipos de redes y sistemas de comunicación.**

f) Se han identificado los **componentes de una red informática**.

g) Se han interpretado **mapas físicos** y **lógicos** de una red informática.

h) Se han reconocido las **normas de seguridad** y **prevención de riesgos laborales** en el uso de los sistemas informáticos.

UT1. EXPLOTACIÓN DE UN SISTEMA INFORMÁTICO: HARDWARE

- Componentes de un sistema informático. Elementos y conceptos fundamentales.
- La información y su representación. Sistemas de numeración y cambios de base. Operaciones lógicas.
- Unidades funcionales. Unidad central de proceso, Memoria y Buses.
- Periféricos. Adaptadores para la conexión de dispositivos.
- Normas de seguridad y prevención de riesgos laborales.

UT1. EXPLOTACIÓN DE UN SISTEMA INFORMÁTICO (II): REDES

- Tipos de redes.
- Componentes de una red informática.
- Topologías de red.
- Mapa físico y lógico de una red local.
- Medios de transmisión. Transmisión alámbrica e inalámbrica.
- Tipos de cableado. Cableado estructurado. Conectores.

COMPONENTES DE UN SISTEMA INFORMÁTICO

¿Qué es un **sistema informático**?

- Desde siempre, las personas han querido crear máquinas que les ayuden a resolver problemas.

- Al principio, cada máquina se diseñaba para una tarea concreta, por ejemplo, un reloj para medir el tiempo o una calculadora para hacer cuentas.

- Es un sistema que permite almacenar y procesar la información.

Pero entonces surgió la pregunta: ¿podemos construir una máquina que sirva para resolver diferentes problemas, no solo uno? La respuesta es sí, y esa máquina se llama sistema informático.

COMPONENTES DE UN SISTEMA INFORMÁTICO (II)

Está formado por tres **componentes**:

- **Hardware:** es la parte física y tangible. Son los componentes físicos del sistema informático: teclado, pantalla, procesador, disco duro...
- **Software:** aplicaciones que permiten operar con los diferentes componentes del sistema, como un juego, un procesador de texto o un navegador de Internet.
- **Personas:** persona o personas que utilizan el sistema informático

Hay autores que añaden:

- **Datos:** la información que el sistema procesa, por ejemplo, los documentos que escribes, las fotos que guardas o los vídeos que ves.

REPRESENTACIÓN Y COMUNICACIÓN DE LA INFORMACIÓN

- Si por algo se distingue al computador como máquina, es que su objeto de procesamiento es la **información**.

- La Informática es, como indica su propio nombre, el **procesamiento de información de forma automática**.

- El propósito de los ordenadores es, por tanto, el almacenamiento, transmisión y procesamiento de la información en general.

- Para que el procesamiento y almacenamiento que éste lleva a cabo es necesario escoger un formato adecuado de representación interna de la información. Igualmente, la transmisión de información requiere un sistema de representación que se adecúe a la tarea,

INFORMACIÓN

- El diccionario de la Real Academia Española de la Lengua define “información” como: “*acción y efecto de informar*”, y como “*comunicación o adquisición de conocimientos que permiten ampliar o precisar los que se poseen sobre una materia determinada*”.

- Podemos decir que un computador es una máquina que procesa datos (“*Información dispuesta de manera adecuada para su tratamiento por un ordenador*”), y los estructura y organiza (*información*), para permitir al ser humano obtener conocimiento (*nueva información*).

SISTEMAS DE NUMERACIÓN

¿Qué es un sistema de numeración?

- Conjunto de reglas, convenios y símbolos que permiten representar números

- Existen diferentes sistemas de numeración:

  - Sistema **decimal** (numeración en base 10): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
  - Sistema **binario** (numeración en base 2): 0, 1
  - Sistema **octal** (numeración en base 8): 0, 1, 2, 3, 4, 5, 6, 7
  - Sistema **hexadecimal** (numeración en base 16): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

SISTEMAS DE NUMERACIÓN (II)

- Podemos definir también un sistema de numeración como un conjunto de dígitos y reglas que permiten **representar datos numéricos**.

- La principal regla es que **un mismo dígito tiene distinto valor según la posición que ocupe**.

TEOREMA FUNDAMENTAL DE LA NUMERACIÓN

- Relaciona una cantidad expresada en cualquier sistema de numeración con la misma cantidad expresada en el sistema decimal; es decir, el valor decimal de una cantidad expresada en otro sistema de numeración que utiliza otra base.

$$N = \begin{cases} \langle d_{(n-1)} \dots d_1 d_0, d_{-1} \dots d_{-k} \rangle = \sum_{i=-k}^{n-1} d_i b^i \\ N = d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{cases}$$

- i = posición respecto a la coma. Para los dígitos de la derecha la i es negativa empezando en -k; para los de la izquierda es positiva empezando en 0.
- k = número de dígitos a la derecha de la coma.
- n = número de dígitos a la izquierda de la coma.
- d = cada uno de los dígitos que componen el número.
- b = base del sistema de numeración (2,8,10 ó 16).

TEOREMA FUNDAMENTAL DE LA NUMERACIÓN (II)

Si tenemos el número 555,55, el dígito 5 tiene distinto valor dependiendo de la posición que ocupa.

Cada posición tiene un peso asociado, siendo en este caso 5 las unidades, 50 las decenas y 500 las centenas. Tambien serán 5 las décimas y 5 las centésimas.

El dígito más a la derecha tendrá peso -2, el siguiente -1, el siguiente 0, y así sucesivamente.

Podremos representar este número como las sumas de las potencias de la base 10 elevada al peso:

$$5 \cdot 10^{-2} + 5 \cdot 10^{-1} + 5 \cdot 10^{0} + 5 \cdot 10^{1} + 5 \cdot 10^{2}$$

EJERCICIO

Obtén el valor de los siguientes números siguiendo el teorema fundamental de la numeración:

327

74,23

TEOREMA FUNDAMENTAL DE LA NUMERACIÓN (III)

Las computadoras suelen efectuar las operaciones aritméticas utilizando una representación para los datos numéricos basada en el sistema de numeración base dos, que se denomina **binario**.

También se utilizan los sistemas de numeración, preferentemente el octal y hexadecimal, para obtener códigos intermedios. Un número expresado en uno de estos sistemas puede transformarse de forma directa a binario y viceversa.

Con ellos se simplifica la transcripción de número binarios y están más próximos al sistema que utilizamos usualmente (sistema digital), por los que a veces se utilizan como paso intermedio en las transformaciones de decimal a binario y viceversa.

EL SISTEMA DECIMAL

El sistema numérico, o BASE10, se utiliza a diario para los cálculos cotidianos, como la contabilidad doméstica, las unidades de medida, las horas, etc.

El sistema decimal utiliza diez dígitos o símbolos: 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9.

Dependiendo de la posición que ocupe un dígito dentro de una cifra, representará las unidades, decenas, centenas, millares, etc. Por esto, se dice que los sistemas de numeración son **posicionales**.

En BASE10 se utilizan las potencias de diez, por ejemplo, 23.605 en BASE10 significa lo siguiente: (2*10000)+(3*1000)+(6*100)+(0*10)+(5*1)

EL SISTEMA BINARIO

El sistema de numeración binario utiliza solo dos dígitos (0 y 1) para representar cantidades, por lo que su base es 2. Cada dígito de un número representado por este sistema se denomina bit (*binary digit*).

Los bits tienen distinto valor dependiendo de la posición que ocupan; por eso este sistema también es posicional.

Estos valores vienen determinados por una potencia de base 2 que la vamos a llamar peso. Así, por ejemplo, el número binario 1011,01 expresado en decimal quedaría así:

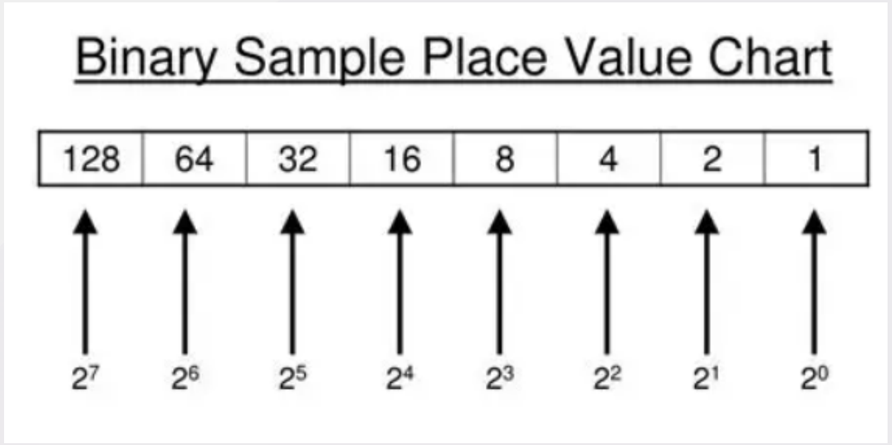
$$1\cdot2^3+0\cdot2^2+1\cdot2^1+1\cdot2^0+0\cdot2^{-1}+1\cdot2^{-2} = 1\cdot8 + 1\cdot2 + 1\cdot1 + 1\cdot0{,}25 = 11{,}25$$

EL SISTEMA BINARIO (II)

Por tanto para pasar de binario a decimal será importante recordar los pesos asociados.

Pesos asociados								Número decimal
2^3	2^2	2^1	2^0	,	2^{-1}	2^{-2}	2^{-3}	
8	4	2	1	,	0,5	0,25	0,125	

Para trabajar con direccionamiento IP más adelante tendrán especial relevancia todos aquellos pesos hasta completar 1 byte (8 bits):



CAMBIOS DE BASE

Al trabajar con diferentes bases se emplea la siguiente notación para indicar que un número está escrito en una base u otra:

- $N_{(10}$ para números en base decimal o base 10.
- $N_{(2}$ para números binarios o en base 2.
- $N_{(8}$ para números octales o en base 8.
- $N_{(16}$ para números hexadecimales o en base 16.

CAMBIOS DE BASE
DE DECIMAL A BINARIO

Para cambiar de binario a decimal, el valor en decimal del número será la suma resultado de cada número multiplicado por la potencia de 2 de la posición que ocupa, tal y como se puede ver en el teorema fundamental de la numeración:

$$N_{(10} = \sum_{i=j}^{k-1} d_i \cdot 2^i$$

Calcula el valor en decimal de los siguientes números en binario.

$100_{(2}$
$1010_{(2}$
$1101_{(2}$
$10011_{(2}$

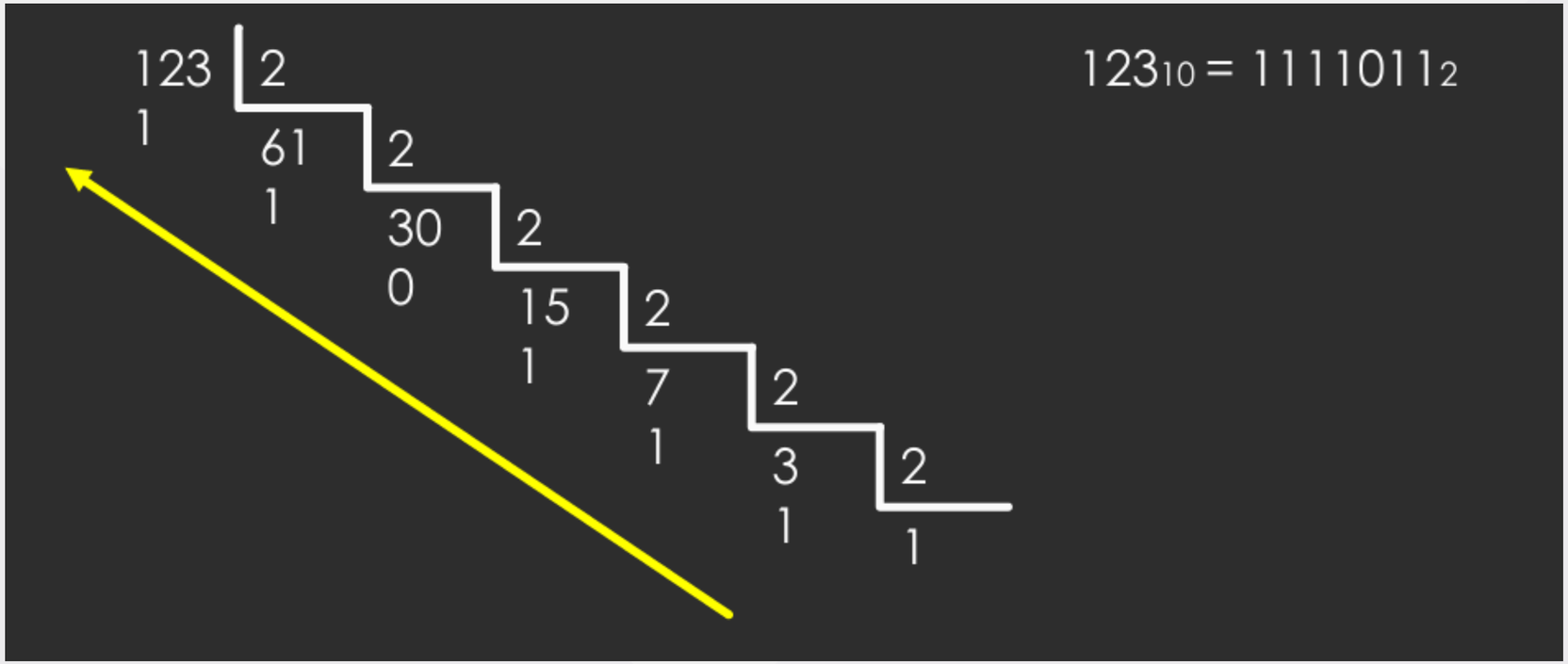
EJERCICIO

Calcula el valor en decimal de los siguientes números en binario:

100
1010
1101
10011

CAMBIO DE BASE DECIMAL A BINARIA

Conversión de 123 a binario:



EL SISTEMA OCTAL

Los primeros sistemas informáticos utilizaban solo el sistema binario para interpretar y transformar los datos, con lo que las labores de programación eran bastante tediosas; se recurrió entonces al uso de sistemas intermedios que permitían una fácil traducción hacia y desde el sistema binario. Estos sistemas son el octal y el hexadecimal.

El sistema octal tiene como base de numeración 8, es decir, utiliza ocho símbolos para representar las cantidades. Estos símbolos son 0, 1, 2, 3, 4, 5, 6, 7. Este sistema también es posicional; o sea, que un mismo dígito tiene distinto valor según la posición que ocupe.

EL SISTEMA HEXADECIMAL

El sistema hexadecimal tiene como base de numeración 16, es decir, utiliza dieciséis símbolos para representar las cantidades. Este sistema también es posicional. A los símbolos A, B, C, D, E y F se les asignan los siguientes valores:

Símbolo	Valor asignado
A	10
B	11
C	12
D	13
E	14
F	15

EL SISTEMA HEXADECIMAL (II)

La numeración en hexadecimal tiene la utilidad de que:

- 4 números binarios se pueden representar directamente con un número hexadecimal
- un byte se representa con dos números hexadecimales

Se utiliza para representar las direcciones de memoria, las direcciones MAC de las tarjetas de red, las direcciones IPv6...

El motivo de que su uso esté tan extendido es que para los seres humanos es más facil y simple leer o escribir valores hexadecimales que números binarios.

CAMBIO DE BASE
DECIMAL A OCTAL Y HEXADECIMAL

Conversión de 123 a octal y hexadecimal:

EQUIVALENCIAS ENTRE SISTEMAS NÚMERICOS

Decimal	Binario	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

OPERACIONES LÓGICAS

En el sistema binario se pueden realizar una serie de operaciones lógicas que se llevan a cabo siguiendo el **álgebra booleana o de Boole**, llamada así en honor al matemático inglés George Boole, que fue el que definió las operaciones lógicas. Internamente un ordenador trabaja con este tipo de lógica.

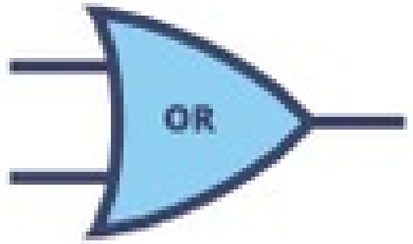
Las operaciones lógicas básicas son:

- **OR:** el resultado de esta operación lógica será 1 cuando alguno de los valores sea 1.
- **AND:** el resultado de esta operación lógica será 1 solo cuando los dos valores sean 1.
- **NOT:** el resultado de esta operación lógica será negar el valor que recibe.

FUNCIÓN LÓGICA BÁSICA OR

El resultado de esta operación lógica será 1 cuando alguno de los valores sea 1.

A	B	OR
0	0	0
0	1	1
1	0	1
1	1	1



FUNCIÓN LÓGICA BÁSICA AND

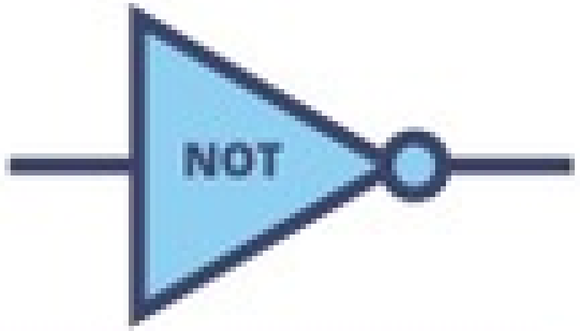
El resultado de esta operación lógica será 1 solo cuando los dos valores sean 1.

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

FUNCIÓN LÓGICA BÁSICA NOT

El resultado de esta operación lógica será negar el valor que recibe.

A	NOT
0	1
1	0



EJERCICIO

Para los siguientes pares de números en binario, realiza las operaciones lógicas OR y AND:

01111100
10001001

OPERACIONES LÓGICAS DERIVADAS

Las operaciones lógicas derivadas son:

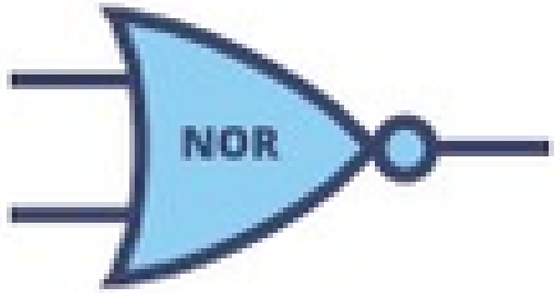
- **NOR:** el resultado de la negación de OR. A NOR B = NOT (A OR B).
- **NAND:** es la negación de AND. A NAND B = NOT (A AND B).
- **XOR:** el resultado de las siguientes operaciones:
  A XOR B = (NOT (A) AND B) OR (A AND (NOT B))).

  El resultado de esta operación lógica será 0 cuando ambos valores sean iguales y 1 cuando sean diferentes.

FUNCIÓN NOR

El resultado de la negación de OR.
A NOR B = NOT (A OR B).

A	B	NOR
0	0	1
0	1	0
1	0	0
1	1	0



FUNCIÓN NAND

Es la negación de AND.
A NAND B = NOT (A AND B).

A	B	NAND
0	0	1
0	1	1
1	0	1
1	1	0

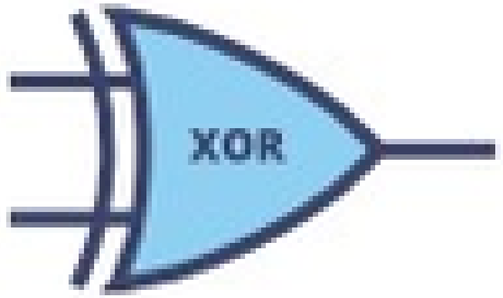
FUNCIÓN XOR

El resultado de las siguientes operaciones:

A XOR B = (NOT (A) AND B) OR (A AND (NOT B))).

El resultado de esta operación lógica será 0 cuando ambos valores sean iguales y 1 cuando sean diferentes.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0



OPERACIONES ARITMÉTICAS: SUMA

Para realizar la operación aritmética suma sería necesario aplicar la siguiente tabla:

A	B	Suma	Acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

El valor que aparece en la columna Acarreo es la cantidad que se arrastra para la operación de los siguientes dígitos a la izquierda.

OPERACIONES ARITMÉTICAS: SUMA (II)

Para sumar los números en binario se puede colocar el acarreo sobre la columna donde se tiene que aplicar.

En el siguiente ejemplo se pueden observar dos sumas de números binarios (junto a cada número y en el resultado se expresa en color azul su valor en decimal):

```
  1111                    11111
  00100110(2    38(10     01101100(2   108(10
+ 10111101(2   189(10   + 10111001(2   185(10
  11100011(2   227(10     100100101(2   293(10
```

EJERCICIO

Suma los siguientes números en binario:

```
  00110111          10101100
+ 10110101        + 00101001
```

OPERACIONES ARITMÉTICAS: RESTA

Para restar en binario se usan los complementos que utiliza el ordenador internamente **para almacenar números negativos**.

Realizar una resta consistiría en sumar el primer número con el complemento del segundo.

Para realizar los complementos se utilizan números de igual longitud y de longitud fija:

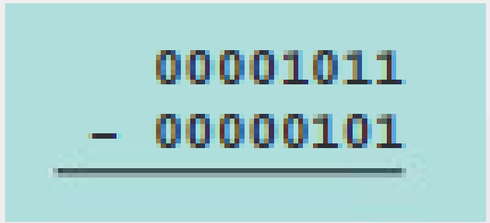
- **Complemento a 1:** sería el resultado de realizar la operación NOT sobre el número
- **Complemento a 2:** sería el resultado de realizar el <<complemento a 1>> sobre el número y sumarle 1 al resultado (solo si el número es negativo).

EJERCICIOS COMPLEMENTOS Y RESTA

- Calcula el complemento a 1 y el complemento a 2 del siguiente número utilizando 8 dígitos:

00001010

- Resta los números binarios utilizando el complemento a 2. Comprueba que los resultados son correctos convirtiendo cada número a decimal.



ALMACENAMIENTO DE LA INFORMACIÓN

Internamente, el ordenador utiliza el **sistema binario** para almacenar en la memoria tanto los **números enteros** y **reales** como los **caracteres**. Existen varios métodos para almacenarlos.

Para **almacenar números enteros** puede utilizar:

- La representación de entero sin signo (almacena un número entero positivo).
- Entero con signo y magnitud (donde el bit más a la izquierda representa el signo, de tal forma que un 0 es positivo y un 1 es negativo.
- Complemento a 1.
- **Complemento a 2.** En realidad es el estándar actual porque no tiene problemas de doble representación de ceros. Así además, la suma y resta funcionan con la misma circuitería, sin necesidad de distinguir el signo

ALMACENAMIENTO DE LA INFORMACIÓN

Almacenamiento de números enteros (II):

Por ejemplo, la representación de un número entero con signo utilizando 4 bytes sería para un número positivo y uno negativo la siguiente:

byte 1	byte 2	byte 3	byte 4	
00000000	00000000	00111101	00111110	(almacena el número $15.678_{(10)}$
10000001	10001011	00100110	01100011	(almacena el número $-25.896.547_{(10)}$

Utilizando 4 bytes, en **entero con signo**, los números mayor o menor que se pueden almacenar son $2^{31} - 1$ y -2^{31}. En entero sin signo el mayor sería $2^{32} - 1$ y el menor 0.

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE

- El almacenamiento de los números en coma flotante se realiza siguiendo el **estándar IEEE 754**.

- Al igual que en la notación científica, se almacenan de la siguiente forma: signo (1 bit), mantisa y el exponente, cuyas longitudes variarán dependiendo de la precisión del número.

- La base será 2 al tratarse del sistema binario, por lo que no será necesario almacenarla.

- El exponente o característica e, se calcula, una vez fijado, con el sesgo en correspondencia con el ancho en bits de este campo. Esta representación sesgada evita tener que almacenar otro bit de signo para el exponente: e=E+S.

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE (II)

- Los números se pueden almacenar en varios bytes, de manera que puede hablarse de precisión simple (4 bytes), doble (8 bytes) y otros formatos.

- En la **precisión simple** (4 bytes): se almacenaría el número utilizando para el signo 1 bit, para el exponente 8 bits (Sesgo = 127) y para la mantisa 23 bits.

- En la **precisión doble** (8 bytes), el signo sería 1 bit, 11 bits el exponente (Sesgo = 1023) y 52 bits la mantisa.

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE (III)

Por ejemplo, en precisión simple, se almacenaría el número $48{,}50_{(10}$ de la siguiente forma:

- Convertimos el número a binario $110000{,}1_{(2.}$
- En binario la notación científica exige que quede como $1{,}\text{mantisa}\times 2^{\text{exponente}}$
- Movemos la coma hasta dejar un único 1 a la izquierda: $110000{,}1_{(2}=1{,}100001\times 2^{5}$
- El exponente sería $5_{(10}$ a este número le tenemos que sumar el sesgo con lo que e=5+127=132 y en binario $10000100_{(2}$. Para el exponente se almacena también el signo, que es el primer bit y es 0 si es negativo y 1 si es positivo.
- La mantisa tiene un tamaño fijo, y si tu número normalizado no ocupa todos los bits, el estándar exige completar el resto con ceros.

signo	exponente	mantisa
0	10000100	10000100000000000000000

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE (IV)

- La mantisa tiene un tamaño fijo, y si tu número normalizado no ocupa todos los bits, el estándar exige completar el resto con ceros.
- En la mantisa, en este formato, se utiliza el bit implícito, que significa que se va a correr la coma hasta el primer 1, pero se deja a la derecha del primer bit a uno. Dicho bit implícito no será representado, porque se sabe que es un 1, con lo que se aumenta en un bit más la precisión de la mantisa.
- El número del ejemplo se almacenaría en 4 bytes de la siguiente forma:

signo	exponente	mantisa
0	10000100	10000100000000000000000

byte 1	byte 2	byte 3	byte 4
01000010	01000010	00000000	00000000

ALMACENAMIENTO DE CARACTERES

Para los caracteres se utilizan los denominados códigos de entrada y salida, o E/S:

- **código ASCII:** 7 bits; los primeros 32 caracteres son de control y los demás representan números, letras y caracteres especiales.
- **ASCII extendido:** 8 bits, que amplió el anterior para añadir caracteres del idioma inglés.
- **ISO-8859-1 o Latin-1:** 8 bits, en el que los 128 primeros caracteres coinciden con la codificación ASCII.
- **ISO-8859-15 o Latin 9:** es el anterior al que se le añadieron algunos caracteres especiales, como el símbolo del €.
- **Unicode:** estándar muy utilizado del que existen tres tipos de codificaciones: UTF-8, UTf-16 y UTF-32
- **y otros...**
- En la actualidad, casi exclusivamente se usa el Unicode 15.1 y UTF-8 como estándar en la Web.

ELEMENTOS FUNCIONALES DE UN ORDENADOR

Un computador u ordenador es una máquina electrónica capaz de procesar datos, aceptarlos desde una unidad de entrada, manipularlos mediante operaciones aritméticas y lógicas, y presentar los resultados a través de una unidad de salida.

Su funcionamiento se basa en un programa almacenado en memoria que controla de forma automática todas las operaciones.

El ordenador digital utiliza un esquema interno de funcionamiento basado en representar la información con un conjunto de valores discretos (normalmente dos valores distintos).

Las operaciones se realizan aplicando una serie de instrucciones sobre los dígitos que representan la información.

EVOLUCIÓN HISTÓRICA: COMPUTADORAS NO DIGITALES

- El **ábaco**, fue una de las primeras herramientas para realizar cálculos aritméticos de forma rápida. Aunque no era un ordenador en el sentido moderno, marcó el inicio del uso de dispositivos para el procesamiento de datos.
- En el siglo XVII, **Blaise Pascal** creó la Pascalina, una calculadora mecánica capaz de sumar y restar automáticamente. Más tarde, en el siglo XIX, **Charles Babbage** diseñó la Máquina Analítica, considerada el primer concepto de ordenador programable, aunque nunca llegó a completarse.
- Paralelamente, se establecieron fundamentos esenciales para la informática actual: el **sistema binario** propuesto por **Leibniz** (siglo XVII), el **álgebra de Boole** (George Boole, siglo XIX), y la **máquina de Turing**, descrita por Alan Turing en 1936 como modelo teórico de computación...

EVOLUCIÓN HISTÓRICA: COMPUTADORAS DIGITALES

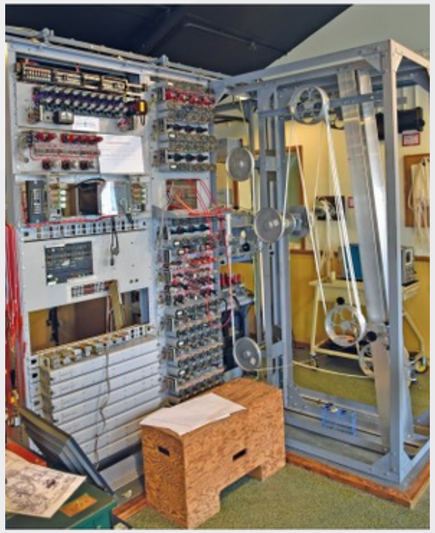
- El uso de la **electricidad** permitió construir ordenadores más rápidos, potentes y fiables que los modelos mecánicos anteriores.

- El primero de este tipo fue el **Z1**, diseñado por el ingeniero alemán Konrad Zuse en 1938. Poco después, en 1939, John Atanasoff y Clifford Berry crearon en EE. UU. el **ABC (Atanasoff-Berry Computer)**.

- Ambos fueron pioneros al usar interruptores eléctricos para representar información: un interruptor apagado indicaba un 0, y uno encendido, un 1.

PRIMERA GENERACIÓN (1946-1955)

- Ordenadores basados en **válvulas electrónicas de vacío**.
- Tenían un tamaño muy grande y su mantenimiento resultaba muy complicado.
- Se destinaban a aplicaciones para el campo científico y militar (Segunda Guerra Mundial: https://www.youtube.com/watch?v=MCljmxr4Ujs).
- Utilizaban como lenguaje de programación el **lenguaje máquina**, no disponían de sistema operativo y empleaban la **tarjeta perforada** para almacenar la información. Ejemplo de estos son el ENIAC y el UNIVAC1.



SEGUNDA GENERACIÓN (1955-1964)

- Se sustituye la válvula de vacío por el **transistor,** lo que influye en que los ordenadores puedan ser más pequeños, generen menos calor y sean más económicos.

- Comenzaron a emplearse lenguajes de alto nivel: Cobol, Algol y Fortran.

- También, memorias de núcleos de ferrita, cintas y tambores magnéticos para almacenar la información. Ejemplo de ordenador (IBM 1401).

- Ejemplo de uso en escenas de cine (Figuras Ocultas): https://www.youtube.com/watch?v=1C5lc2GIisQ

TERCERA GENERACIÓN (1964-1974)

- Ordenadores basados en **circuitos integrados** (en un solo chip se integran varios transistores). Esto supuso la minimización de los ordenadores, así como el aumento notable de la velocidad.
- En esta época evolucionó el software de forma considerable, sobre todo en que se desarrollan los **sistemas operativos,** en los que se incluyó la multiprogramación y el tiempo real.
- También evolucionaron apreciablemente las unidades de almacenamiento, y aparecieron los discos magnéticos.
- Comenzaron a utilizarse memorias de semiconductores...
- Ejemplo de ordenador (IBM 370).

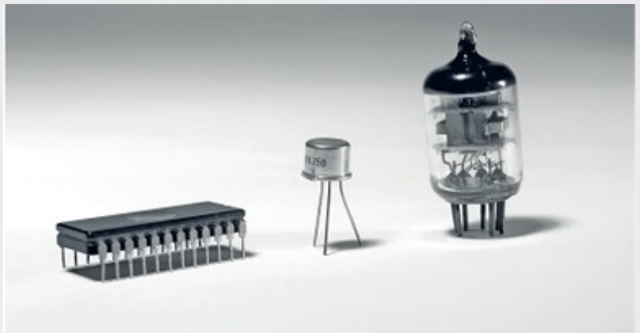


Figura 1.9. *Comparación del tamaño de un chip, un transistor y un tubo de vacío.*

CUARTA GENERACIÓN (1974-1983)

- La componen los ordenadores que presentan toda la CPU en un solo circuito integrado: los **microprocesadores**.
- El primer micro (el 4004) lo desarrolló Intel en 1971.
- Continúa la reducción del tamaño y del coste de los ordenadores
- Empieza a utilizarse el ordenador para uso personal o doméstico.
- Surgen los sistemas operativos CP/M y Unix, precursor del actual Linux.
- En esta generación hacen su aparición las **aplicaciones de uso general y de gestión**.

ESCALAS DE INTEGRACIÓN

Dependiendo del número de transistores en un chip se habla de diferentes escalas de integración:

Nivel de integración	Componentes integrados en un chip
SSI (*Small Scale Integration*)	10-100
MSI (*Medium Scale Integration*)	101-1.000
LSI (*Large Scale Integration*)	1.001-10.000
VLSI (*Very Large Scale Integration*)	10.001-100.000
ULSI (*Ultra Large Scale Integration*)	100.001-1.000.000
GLSI (*Giga Large Scale Integration*)	1.000.001 o más

QUINTA GENERACIÓN (DESDE 1983)

- En 1983, Japón lanzó el llamado Programa de la Quinta Generación de Computadoras, con los objetivos explícitos de producir máquinas capaces de comunicarse en un l**enguaje más cotidiano** y no a través de códigos o lenguajes de control especializados.
- Hay autores que ponen el comienzo en 1981 con la comercialización del PC (Personal Computer) por parte de IBM.
- Surgen los sistemas operativos MS-DOS, Windows, Linux y Mac OS.
- Se desarrolla la programación orientada a objetos (POO) y surgen lenguajes como Java, C++, C# y Delphi, entre otros.

QUINTA GENERACIÓN (DESDE 1983) (II)

- Esta generación sigue hasta nuestros días

- Parece que la próxima generación será la que venga marcada por el desarrollo y la generalización de los ordenadores cuánticos.

CLASIFICACIÓN DE LOS SISTEMAS INFORMÁTICOS: ATENDIENDO AL USO

- **Sistemas informáticos de uso general:**
  - Se utilizan para una amplia gama de funciones y utilidades.
  - Usan muchas aplicaciones de diversos tipos.
- **Sistemas informáticos de uso específico:**
  - Se utilizan para una tarea concreta: control de tráfico o hardware específico (un coche, una televisión o cualquier electrodoméstico).
  - Normalmente se fabrican para el fin para el que se han diseñado.

CLASIFICACIÓN DE LOS SISTEMAS INFORMÁTICOS: ATENDIENDO AL TAMAÑO

- Superordenadores
- Macroordenadores o *mainframes*
- Servidores y estaciones de trabajo (*workstations*)
- Ordenadores personales
- Clientes ligeros (*thin clients*)

SUPERORDENADORES

- Son equipos que se fabrican por grandes corporaciones con fines científicos, militares, tecnológicos...
- No son equipos que estén en el mercado, sino que se construyen especialmente para el fin por el que se han diseñado.
- Tienen un gran rendimiento, potencia y capacidad.
- Su rendimiento se suele medir en petaflops (número de operaciones de coma flotante que pueden realizarse por segundo).
- Se les conoce por un nombre propio que los identifica de los demás. En esta web podéis encontrar un ranking: https://www.top500.org/

SUPERORDENADORES (II)

Entrada al superordenador Mare Nostrum situado en el Barcelona Supercomputing Center (BSC).

MACROORDENADORES O MAINFRAMES

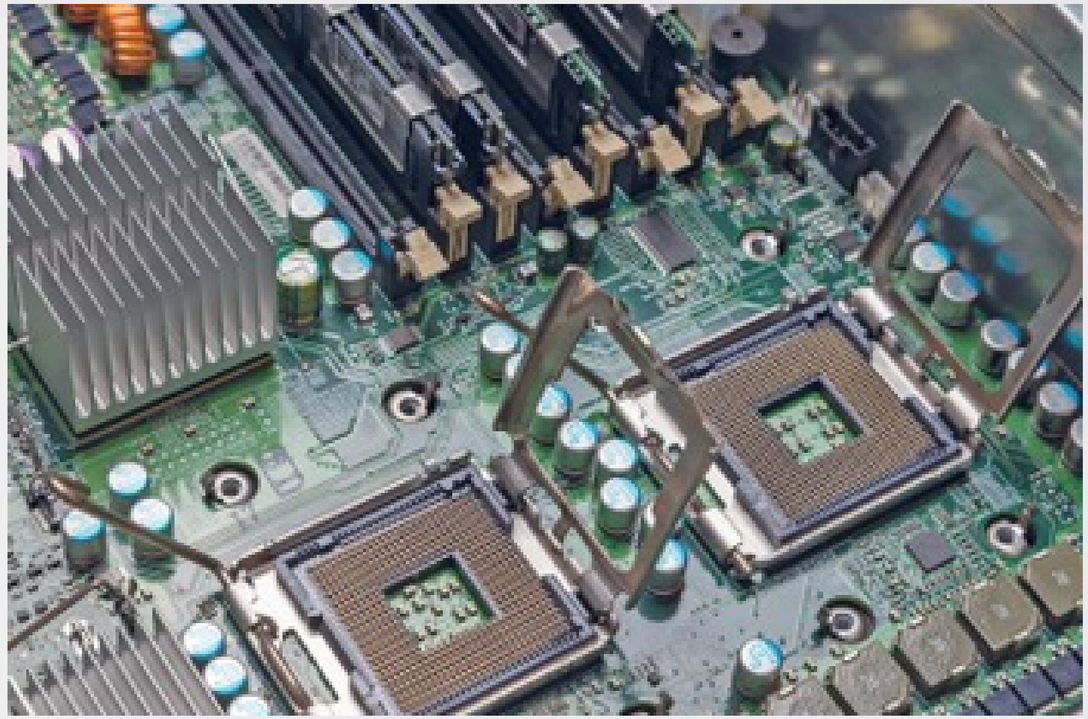
- Ocupan grandes espacios y dan servicio a muchos usuarios que pueden acceder a ellos a través de la red.
- Tienen un gran tamaño, gran capacidad de almacenamiento y grandes prestaciones.
- Deben estar en habitaciones preparada para mantener su funcionamiento en condiciones ambientales óptimas de temperatura, humedad...

SERVIDORES Y ESTACIONES DE TRABAJO

- Se utilizan para prestar servicios a distintos usuarios y a otros equipos de una red.

- Pueden ofrecer acceso a los recursos del sistema, acceso a internet, a diferentes aplicaciones.

- Necesitan una alta capacidad de proceso para poder atender las peticiones de los usuarios.



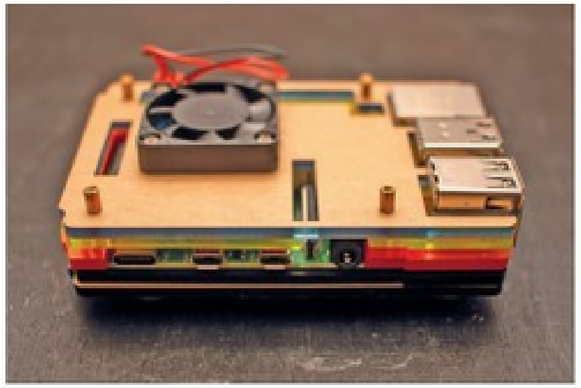
ORDENADORES PERSONALES

- Son equipos utilizados para el ámbito doméstico y profesional.

- Se pueden clasificar en:
  - **Sobremesa:** se utiliza para tenerlo en un lugar fijo (trabajo, aula, hogar...)
  - **Portátil:** también llamado *laptop* o *notebook*, dependiendo del tamaño.
  - **Convertible 2 en 1:** tiene pantalla táctil y abatible para poder ser usado como *tablet* y se le puede incoroporar fácilmente ratón y teclado para ser usado como portátil.
  - ***Tablets*:** dispositivo de mayor tamaño que un *smartphone* también con una pantalla táctil. Utiliza los mismos sistemas operativos como (iOS o Android) y tiene la mayoría de sus funciones, salvo la de realizar llamadas.
  - ***Smartphone:*** teléfono móvil con sistema operativo y funciones adicionales (reproductor multimedia, cámara, GPS...).
  - **Ordenador de una sola placa (SBC, *Single Board Computer*)**

ORDENADOR DE UNA SOLA PLACA (SBC)

- Ordenador completo construido en un solo circuito impreso que contiene toda la funcionalidad de un ordenador, pero de tamaño reducido.
- Un ejemplo de este tipo de ordenador son las Rapsberry Pi.
- Son muy versátiles y pueden utilizare para muchos usos.
- También pueden ser utilizados como clientes ligeros para conectarse a un servidor.

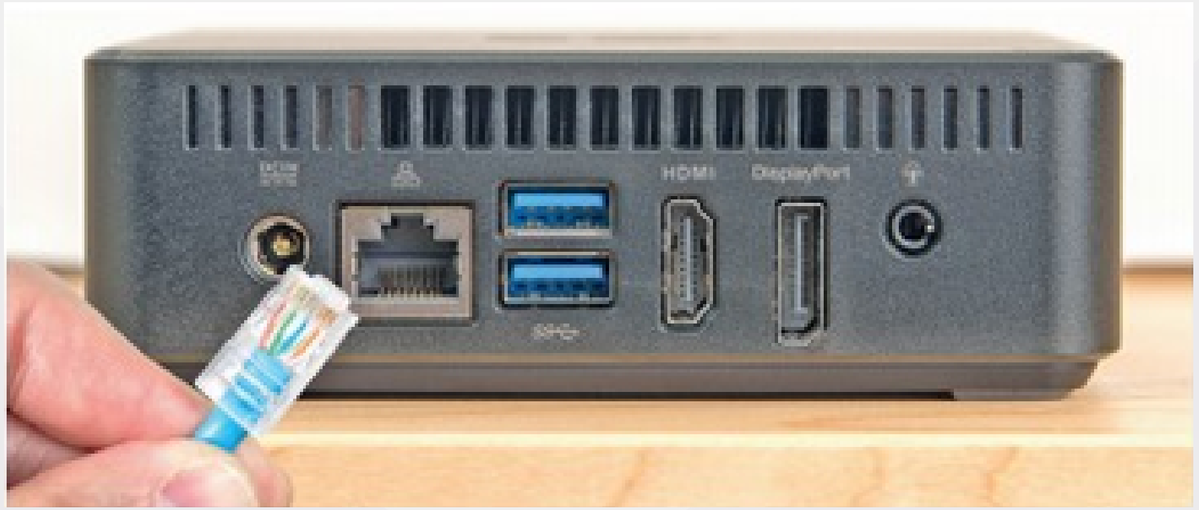


NOTA

- Los ordenadores personales pueden tener distintas capacidades dentro de un mismo tipo. Entre ellos están los ordenadores de tipo ***gaming*** que tienen altas prestaciones de memoria, procesamiento y tarjeta gráfica.

CLIENTES LIGEROS (THIN CLIENTS)

- Son ordenadores que se utilizan principalmente para conectarse a un servidor.
- Tiene poca capacidad y poca necesidad de almacenamiento y procesamiento.



ARQUITECTURA DE UN ORDENADOR

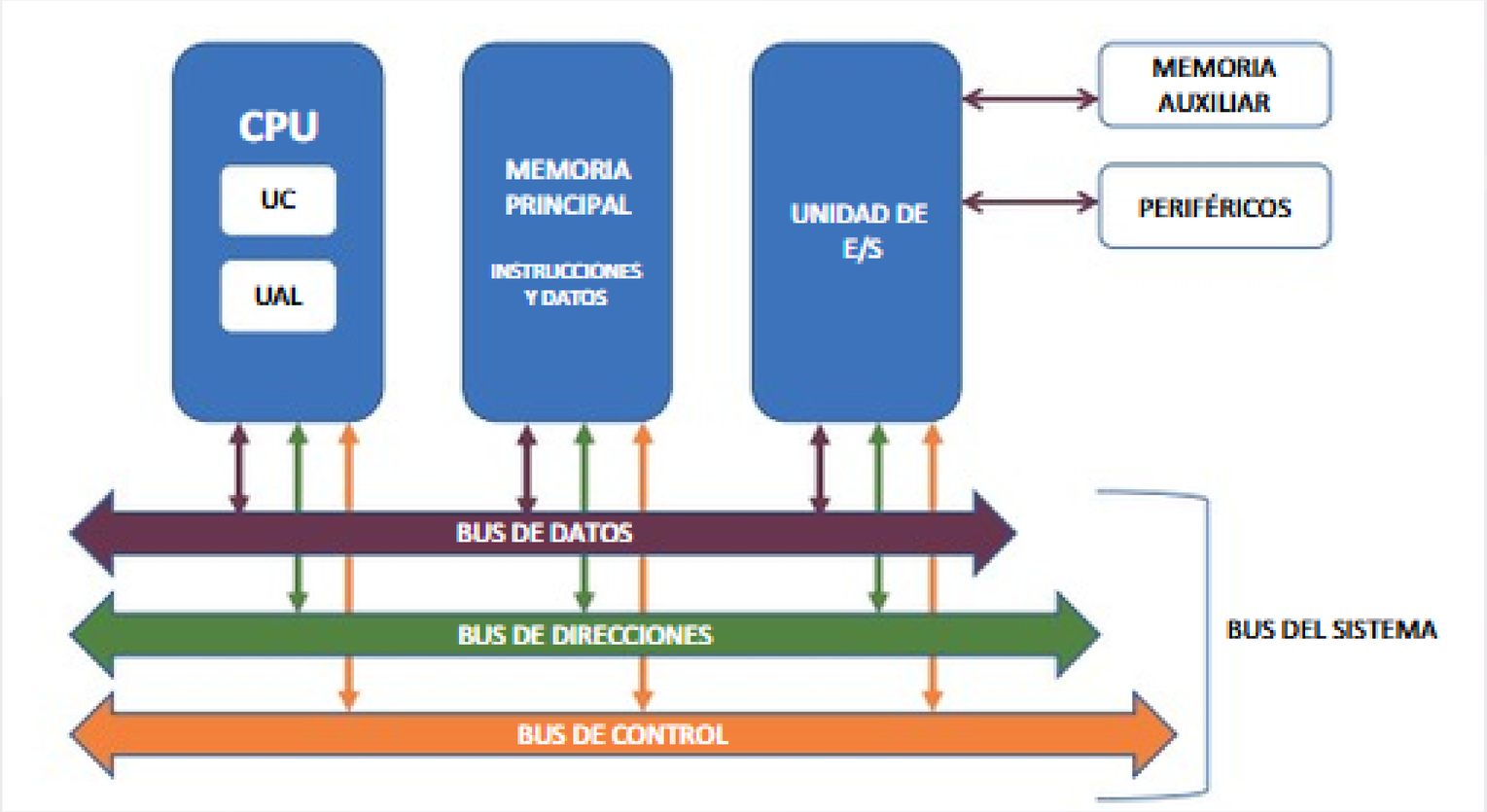
- Se trata de un modelo para diseñar teóricamente un sistema informático.

- Los dos tipos más extendidos son:

  - **Arquitectura de von Neumann:** esta arquitectura fue desarrollada por John von Neumann en 1945.

  - **Arquitectura de Harvard:** se desarrolló para el ordenador Harvard Mark I (1944). Se diferencia del modelo anterior en que se separan las memorias de datos y de instrucciones, así como el bus de datos, que se divide en bus de datos y en bus de instrucciones. Existe una arquitectura Harvard modificada que no es tan estricta en la separación entre instrucciones y datos.

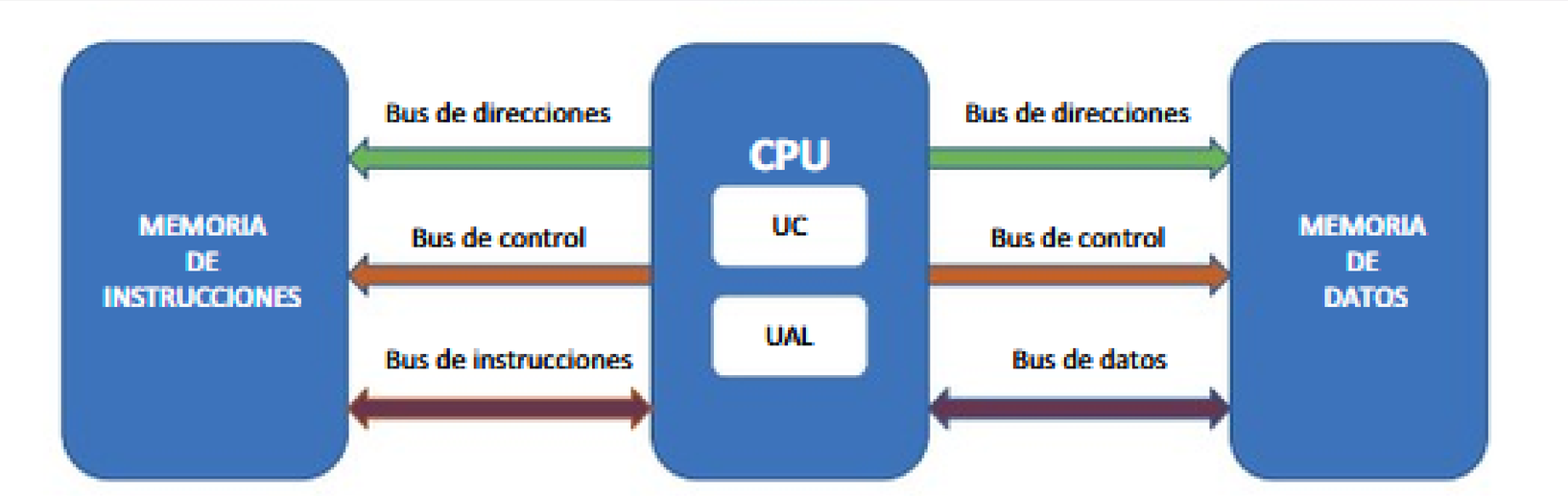
ARQUITECTURA DE VON NEUMANN



ARQUITECTURA DE VON NEUMANN (II)

- Funcionamiento básico de la arquitectura Von Neumann:
  - Para ejecutar un programa, primero hay que buscarlo en la memoria principal.
  - Una vez localizado el programa, se carga en la CPU instrucción a instrucción.
  - En la CPU se realizan todas las operaciones del programa.
  - Los resultados se envían a la memoria principal para ser almacenados.

ARQUITECTURA HARVARD



- En este modelo, las instrucciones y los datos se almacenan en cachés diferentes para mejorar el rendimiento.

- Tiene el inconveniente de tener que dividir la cantidad de memoria caché entre los dos, por lo que funciona mejor sólo en los casos particulares cuando la frecuencia de lectura de instrucciones y de datos es aproximadamente la misma

ARQUITECTURAS HÍBRIDAS

Combinan características de distintos modelos clásicos y modernos:

- Las arquitecturas que **integran principios de Von Neumann y Harvard** (muchos procesadores ARM), donde se separan ciertos caminos de datos e instrucciones para mejorar el rendimiento.

- **Arquitecturas heterogéneas CPU+GPU**, distintos tipos de procesadores colaboran para repartir las tareas según su especialización. Los GPU dedicados al procesamiento paralelo masivo para gráficos, cómputo científico, entrenamiento de IA.

ARQUITECTURAS HÍBRIDAS (II)

- **Arquitecturas orientadas a inteligencia artificial**: incorporan unidades específicas como tensor cores (NVIDIA, unidades dentro de las GPU diseñadas para acelerar operaciones de matrices con precisión mixta) o NPUs (*Neural Processing Units*) para acelerar operaciones de aprendizaje automático, y, en un plano todavía emergente.

- **Arquitecturas cuánticas:** representan un nuevo paradigma de computación basado en qubits y principios de la mecánica cuántica.

VON NEUMANN EN LA ACTUALIDAD

Con el tiempo, la arquitectura de Von Neumann ha sido mejorada con técnicas como:

- la **E/S mapeada en memoria**, que permite tratar dispositivos como posiciones de memoria.
- la arquitectura Harvard (separación de instrucciones y datos).
- la **segmentación de instrucciones** (*pipelining*).
- el uso de **memoria caché**.
- **procesadores multinúcleo**.
- **ejecución fuera de orden**.

ELEMENTOS FUNCIONALES

Según el modelo de Von Neumann, todo ordenador debe tener al menos estos **cuatro elementos**:

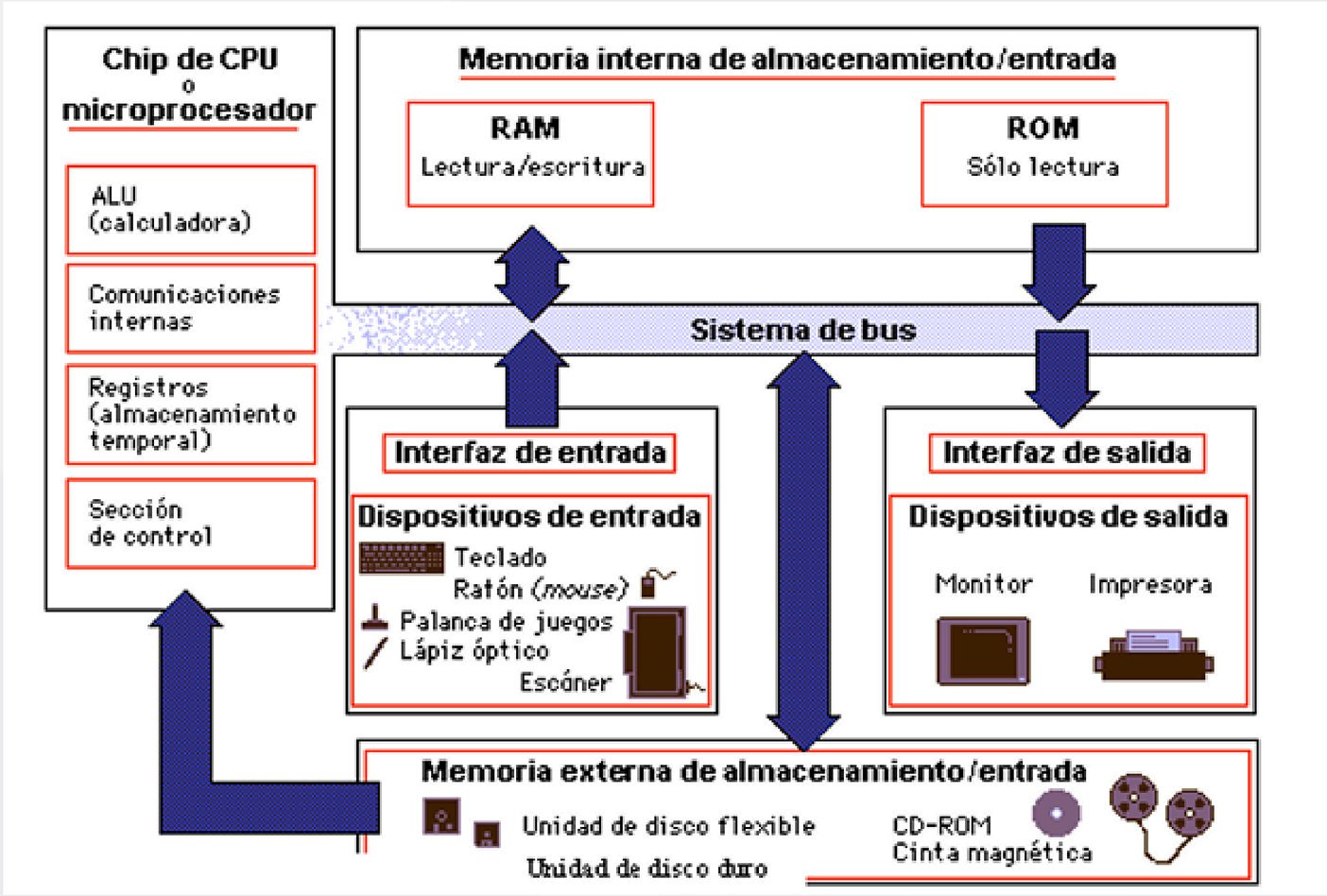
1. CPU (Unidad Central de Proceso): Es el “cerebro” del ordenador. Se encarga de interpretar y ejecutar las instrucciones.

2. Memoria principal: Almacena tanto los datos como las instrucciones del programa (algo que diferencia esta arquitectura de Harvard.

3. Buses: Son los “cables virtuales” que transportan información entre los componentes: datos, direcciones e instrucciones.

4. Dispositivos de entrada/salida (E/S): Permiten comunicarse con el exterior: teclado, pantalla, impresora, ratón, etc.

ELEMENTOS FUNCIONALES (II)



UNIDAD CENTRAL DE PROCESO

- Se llama **Unidad central de proceso** o **UCP (CPU**, ***C****entral* ***P****rocess* ***U****nit*) o microprocesador.

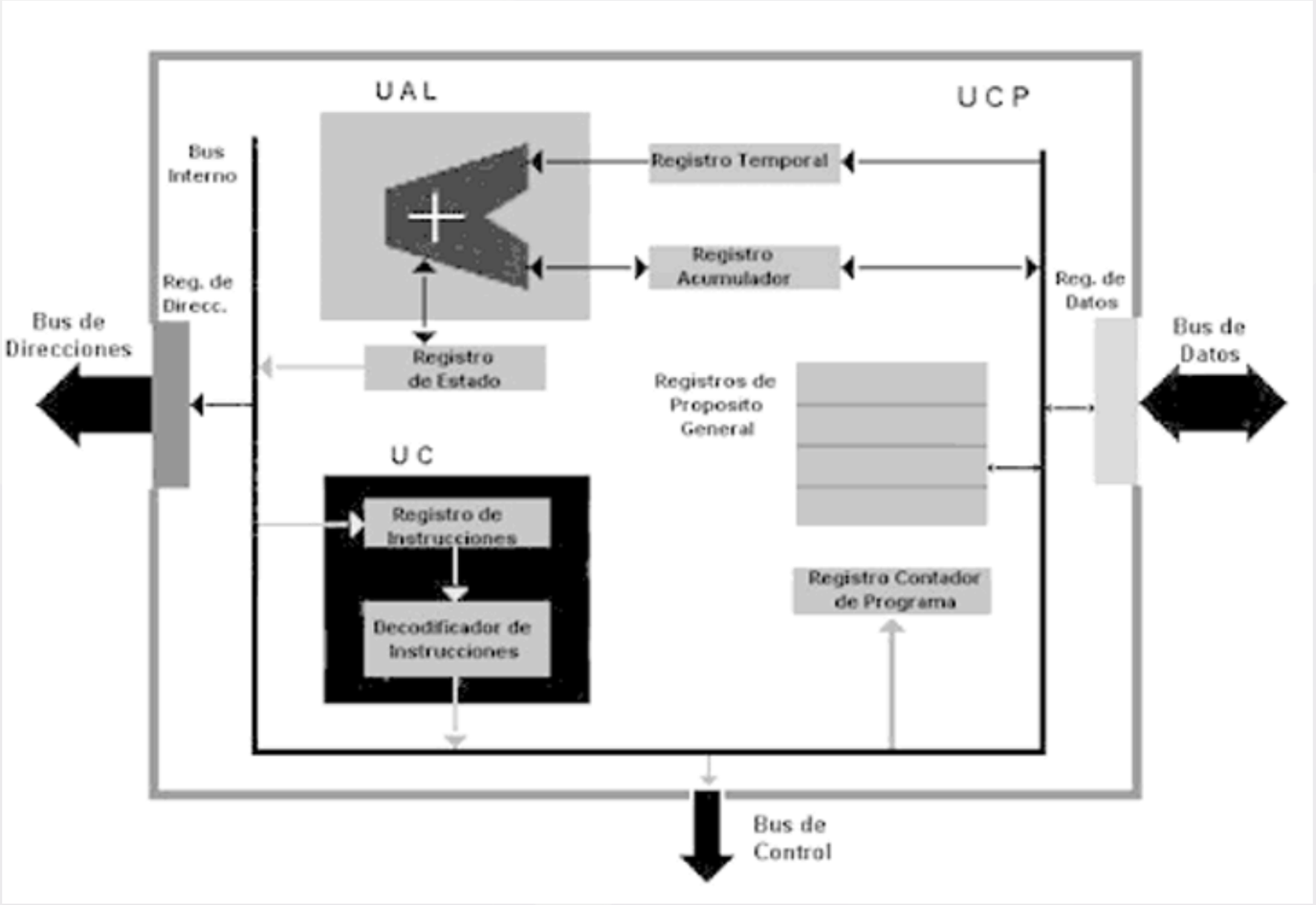
- Se considera el componente principal del ordenador, ya que controla todo su funcionamiento enviando señales.

- Es el “cerebro del ordenador”

- Controla qué elemento es el que debe entrar en funcionamiento y realiza las operaciones que se realizan en el ordenador.

- Contiene varios elementos, entre los que destacan la UAL (Unidad Aritmético Lógica) y la UC (Unidad de Control) y los registros de trabajo o de propósito general.

UNIDAD CENTRAL DE PROCESO (II)



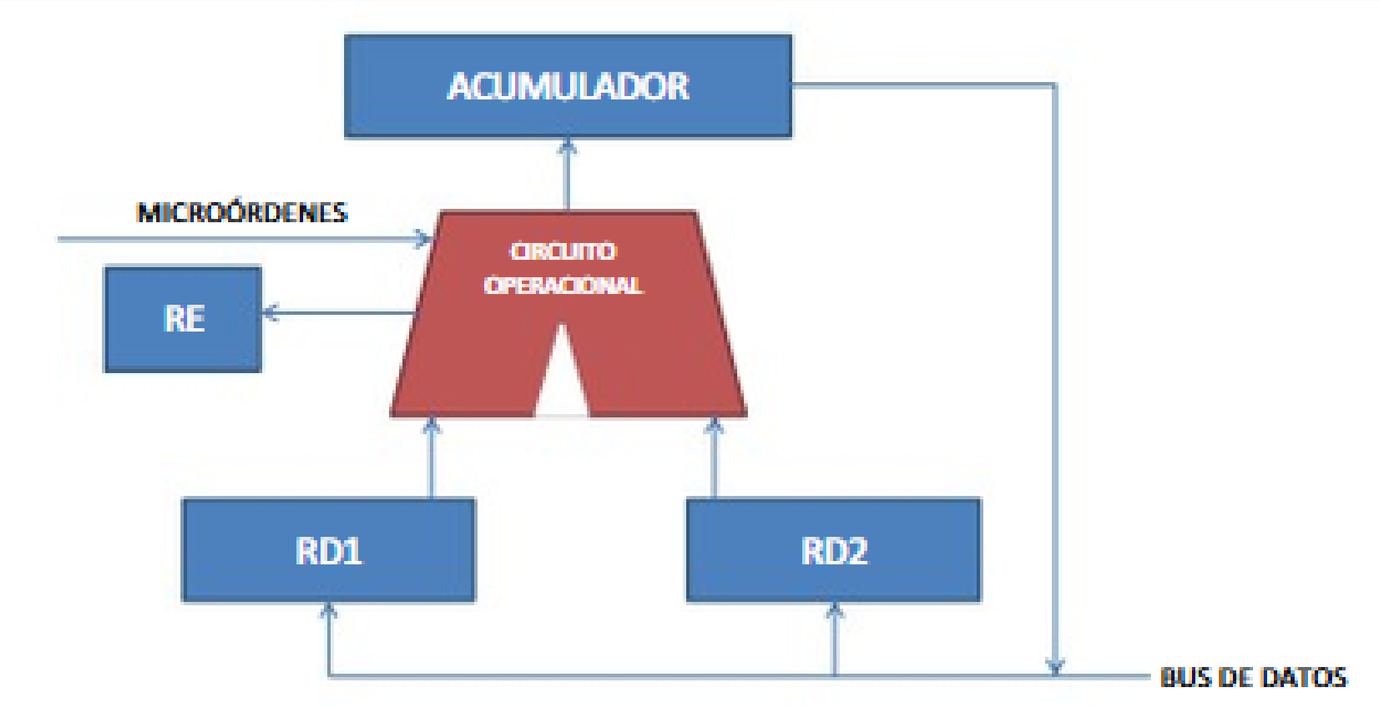
ESTRUCTURA DE LA CPU

- **Unidad de control (UC):** interpreta y ejecuta las instrucciones máquina almacenadas en la memoria principal y genera las señales de control necesarias para ejecutarlas.

- **Unidad aritmético-lógica (UAL, o ALU):** recibe los datos sobre los que efectúa operaciones de cálculo y comparaciones, toma decisiones lógicas (determina si una afirmación es cierta o falsa mediante las reglas del álgebra de Boole) y devuelve luego el resultado; todo ello bajo la supervisión de la unidad de control.

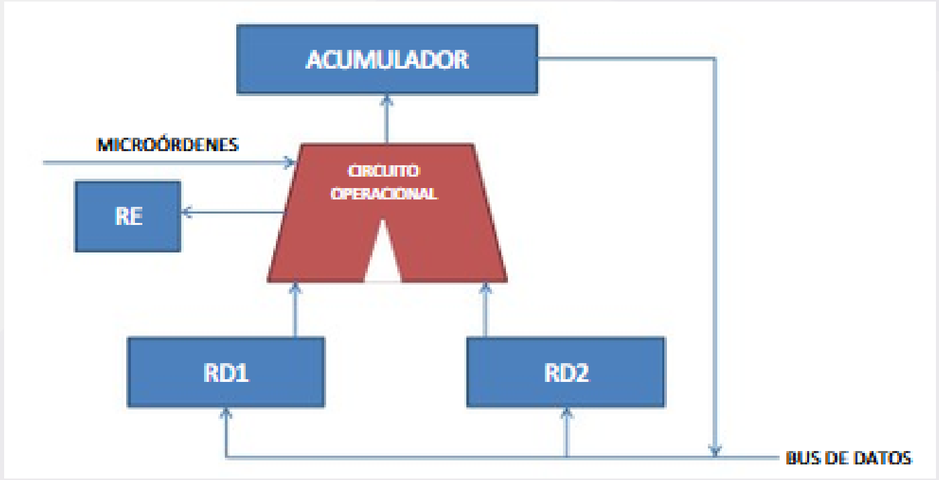
- **Registros de trabajo o de propósito general:** donde se almacena información temporal, que constituyen el almacenamiento interno de la CPU.

ESTRUCTURA DE LA ALU



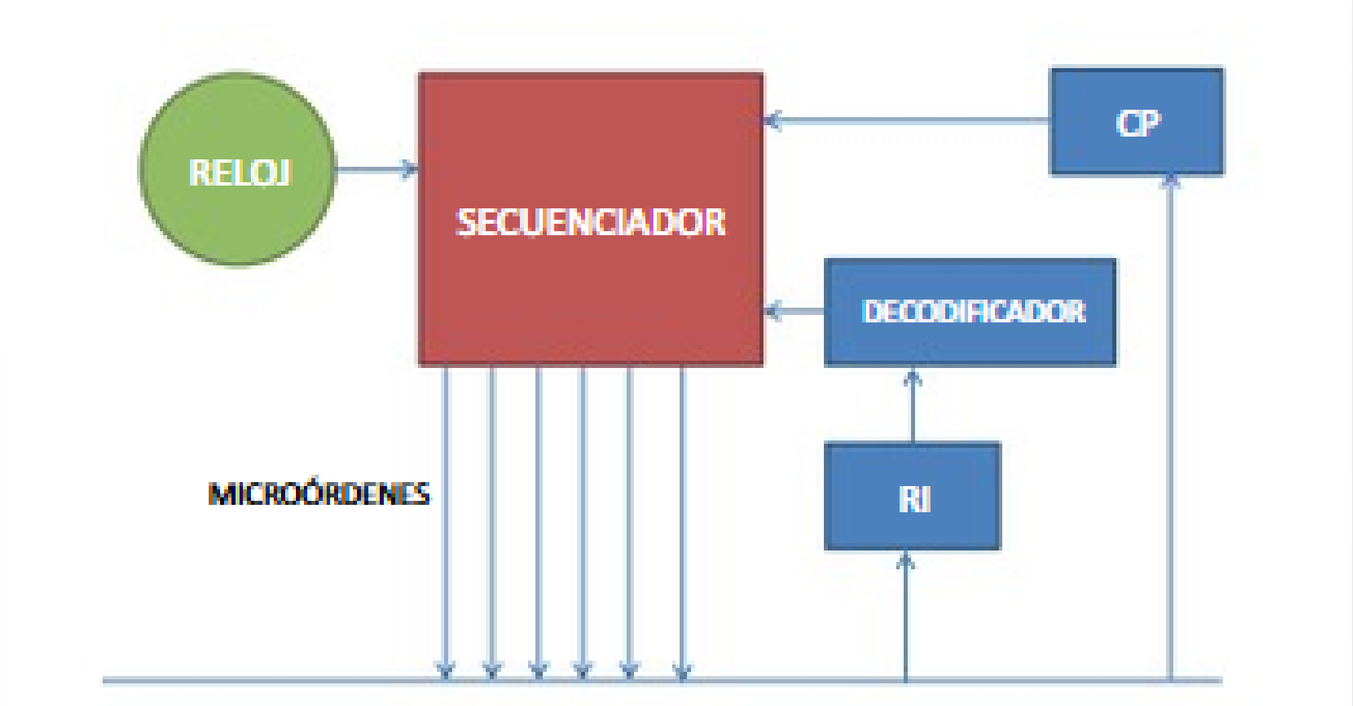
ESTRUCTURA DE LA ALU (II)

- **Registro de datos (RD):** son los datos de entrada u operandos.
- **Registro acumulador (AC):** almacena el resultado de la última operación realizada.
- **Registro de estado (RE):** almacena las condiciones de la última operación, Z (el resultado fue cero), S (el resultado fue negativo), C (hubo acarreo), 0 (hubo desbordamiento)...
- **Circuito operacional:** realiza las operaciones aritméticas o lógicas.



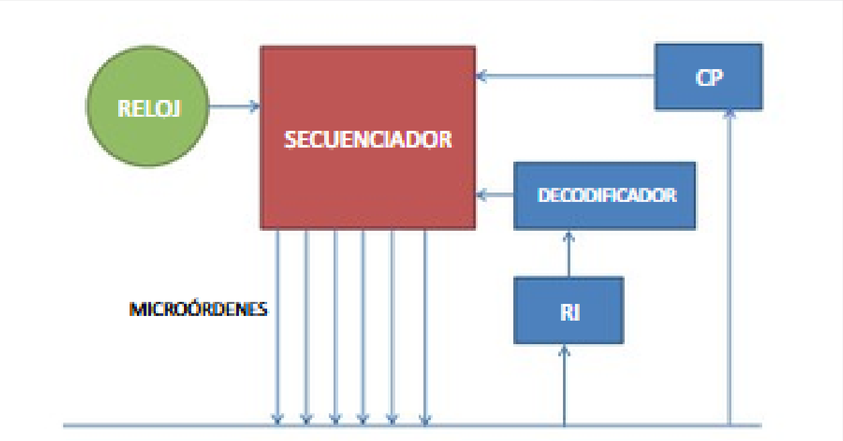
ESTRUCTURA DE LA UC

Envía señales al resto de los elementos del ordenador para indicar cuál es el que debe ponerse en funcionamiento en cada momento. Las señales las envía a través del bus de control.



ESTRUCTURA DE LA UC

- **Registro de instrucción (RI o IR):** contiene la instrucción que se está ejecutando.
- **Registro contador de programa (CP):** almacena la dirección de memoria donde está la siguiente instrucción que se ha de ejecutar.
- **Decodificador:** interpreta la operación del registro de instrucción (RI).
- **Generador de señales o secuenciador:** sincronizado por el reloj, genera microórdenes para que se ejecute la instrucción almacenada en el registro de instrucción (RI).
- **Reloj:** genera las señales para controlar y sincronizar el resto del sistema.



MEMORIA PRINCIPAL

En esta memoria se cargan los datos y las instrucciones que debe ejecutar el microprocesador.

Está formada por múltiples celdas, cada una almacenando un único valor binario: 0 ó 1 (1 bit).

Un conjunto de celdas de tamaño fijo se denomina **palabra**. Los ordenadores suelen usar palabras de 32 o 64 bits. Cada palabra tiene una dirección única asociada. Al leer información de la memoria, se accede siempre a la palabra completa ubicada en la dirección seleccionada.

Por ejemplo, una memoria de 192 bits se organiza en 6 palabras de 32 bits cada una. Estas palabras se numeran con direcciones del 0 al 5. La primera palabra incluye los bits del 0 al 31, la segunda del 32 al 63, y así sucesivamente.

RAM

La memoria principal se denomina **RAM** (**R**andom **A**ccess **M**emory, memoria de acceso aleatorio).

La RAM es volátil, lo que significa que necesita estar alimentada con energía para mantener la información; si se corta la alimentación, los datos se pierden.

Además, el tiempo que tarda en leer o escribir una palabra es constante, independientemente de su posición en la memoria.

MEMORIAS VOLÁTILES

- **Registros:** Pequeñas celdas de memoria muy rápidas ubicadas dentro de la CPU donde se almacenan datos e instrucciones inmediatas necesarias para realizar operaciones con gran rapidez.

- **Memoria caché:** Una jerarquía de memorias rápidas (niveles L1, L2 y L3) que almacenan datos usados frecuentemente para acelerar el procesamiento y reducir el tiempo de acceso a la RAM.

- **RAM (Memoria de Acceso Aleatorio):** Memoria volátil de mayor tamaño y menor velocidad que la caché, donde se cargan los programas y datos que el sistema está ejecutando en cada momento.

OTROS TIPOS DE MEMORIAS

- **ROM (Memoria de Solo Lectura):** Es una memoria de solo lectura que no puede borrarse. Se usaba comúnmente en BIOS antiguas para almacenar el firmware de forma permanente.

- **EPROM (Memoria de Solo Lectura Programable y Borrable):** Es una memoria no volátil que es de solo lectura, pero puede borrarse mediante luz ultravioleta y reescribirse. Se usaba en BIOS donde los datos se leen con frecuencia, pero se reescriben raramente.

- **EEPROM (Memoria de Solo Lectura Programable y Borrable Eléctricamente):** Similar a la EPROM, pero puede borrarse eléctricamente y reprogramarse sin necesidad de retirarla. Se usa en memorias BIOS que requieren actualizaciones ocasionales.

OTROS TIPOS DE MEMORIAS (II)

- **Memoria Flash:** Evolución de la EEPROM que permite leer y escribir en múltiples posiciones simultáneamente. Se usa habitualmente en memorias USB y es la tecnología más común para almacenar BIOS modernos, especialmente UEFI.

- **Memoria de firmware (BIOS vs UEFI):**
  - El firmware BIOS tradicional usaba ROM o EEPROM con capacidad limitada (16 bits, 1 MB direccionable) y sin soporte para características modernas (seguridad).

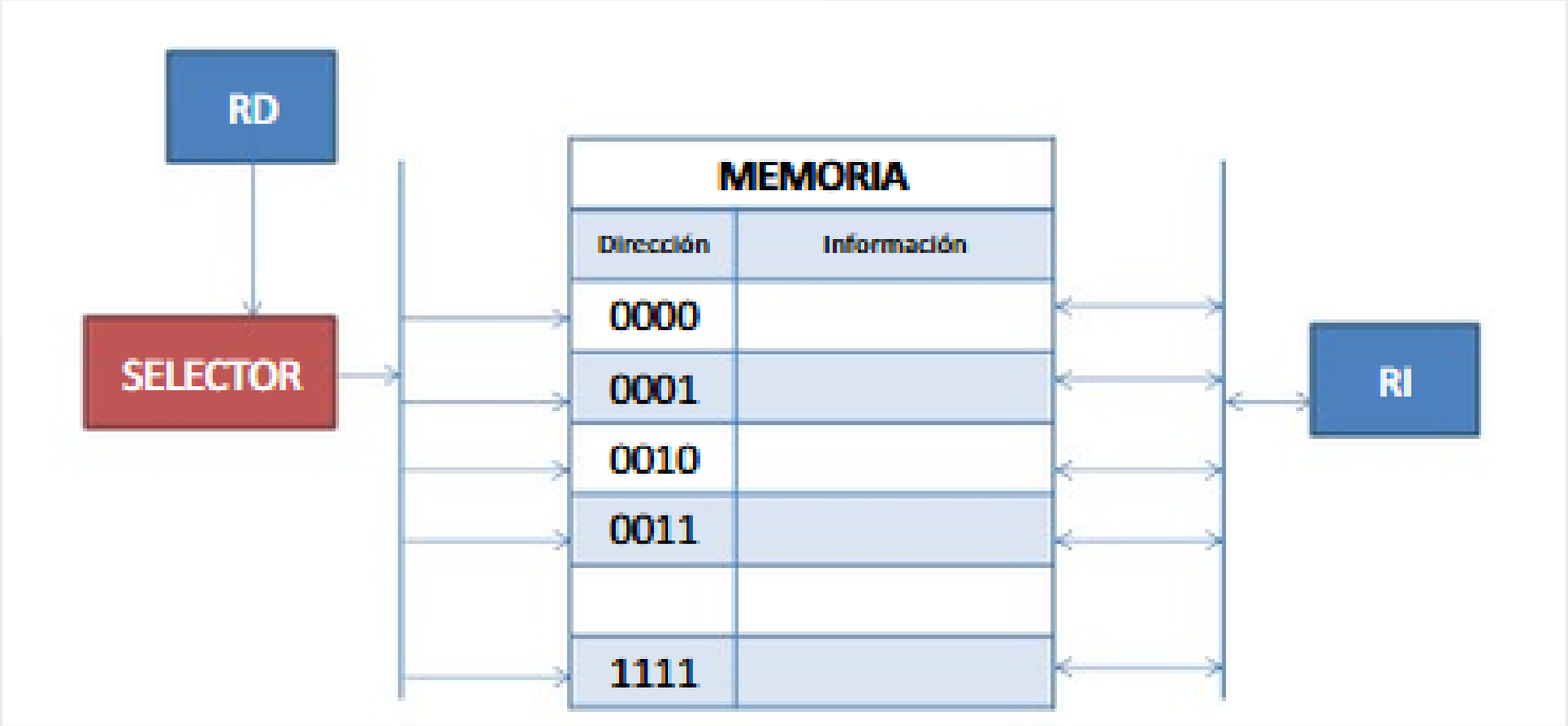
  - El firmware UEFI actual utiliza memoria Flash NOR (entre 4 y 32 MB), que permite ejecutar código directamente desde la memoria (XIP), ofrece mayor fiabilidad, arquitectura modular y soporta funciones avanzadas como Secure Boot y discos grandes.

MEMORIA PRINCIPAL (II)

Dentro de la memoria se encuentran una serie de elementos:

- **Registro de direcciones (RD):** almacena la dirección de memoria sobre la que se va a realizar una operación de lectura o escritura.
- **Registro de intercambio (RI):** contiene el dato que va a ser escrito o leído de la memoria.
- **Selector de memoria (SM):** selecciona la dirección de memoria que se encuentra en el registro de direcciones (RD) y, según sea una operación de lectura o escritura, lee el dato de la memoria y lo almacena en el registro de intercambio (RI) o lee el dato del registro de intercambio (RI) y lo almacena en la memoria, respectivamente.

MEMORIA PRINCIPAL (III)



OTROS ELEMENTOS FUNCIONALES

- **Unidad de E/S:** Esta unidad se encarga de la comunicación entre la CPU o microprocesador y los componentes externos como los periféricos y las unidades de almacenamiento externo.

- **Periféricos:** Se denomina periférico a cualquier dispositivo externo que permite introducir u obtener información de un ordenador. Los hay de entrada, de salida y de entrada/salida.

Los periféricos suelen conectarse al ordenador mediante diferentes interfaces o buses, como USB, HDMI o PCIe, que permiten la comunicación rápida y eficiente entre el sistema y estos dispositivos.

BUS DEL SISTEMA

- **Bus del sistema:** Todos los elementos se conectan entre sí a través del bus del sistema. Este bus lleva la información entre los diferentes elementos del sistema.

  Dependiendo de la información que se lleva se diferencian tres tipos de buses:

  - **Bus de datos:** transporta la información, es decir, los datos y las instrucciones entre los componentes del ordenador.

  - **Bus de direcciones:** se encarga de llevar la dirección de la memoria donde se va a almacenar o a leer su contenido.

  - **Bus de control:** envía desde la unidad de control señales a todos los elementos del ordenador para indicar cuál de ellos es el que tiene que entrar en funcionamiento.

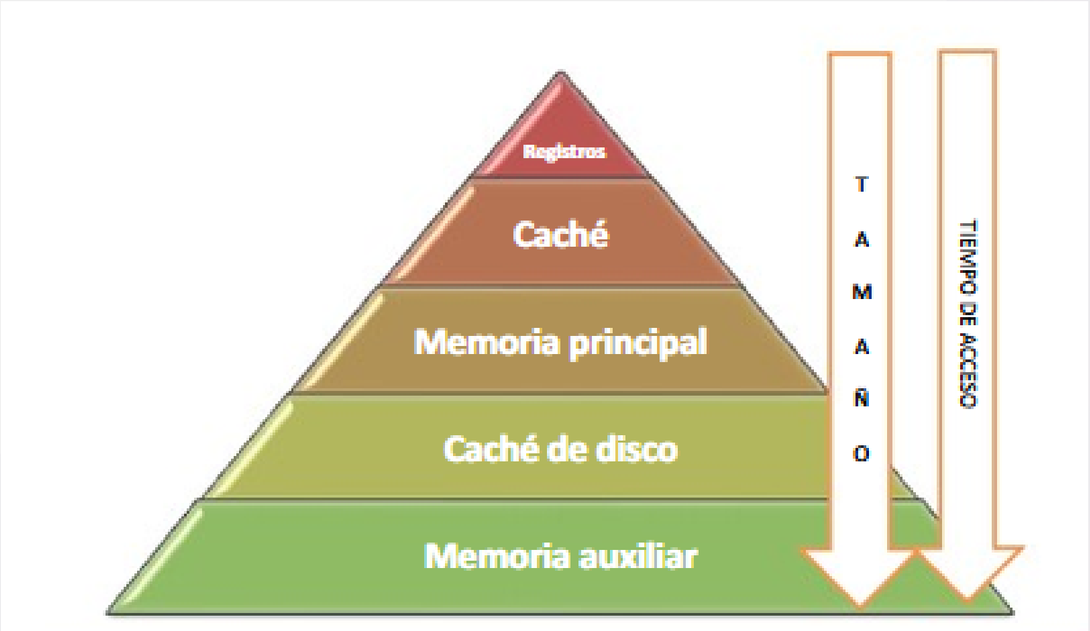
MEMORIAS AUXILIARES

- **Bus del sistema o dispositivos de almacenamiento externo:**
  - Son dispositivos con mayor capacidad que la memoria interna.
  - Almacenan información de forma permanente, es decir, no son volátiles.
  - Tienen también un mayor **tiempo de acceso** que la memoria principal.

JERARQUÍA DE MEMORIA

- Las unidades de almacenamiento se pueden clasificar atentiendo a su tamaño y a su tiempo de acceso. Esto se conoce como **jerarquía de memoria**.

- Cuanto más arriba estén en la clasificación, menor será el tiempo de acceso y la capacidad o el tamaño.



MEMORIA CACHÉ

Una jerarquía de memorias rápidas (niveles L1, L2 y L3) que almacenan datos usados frecuentemente para acelerar el procesamiento y reducir el tiempo de acceso a la RAM.

- **L1:** normalmente integrada en cada núcleo y muy pequeña. Es la más rápida.
- **L2:** también suele estar integrada en el procesador, generalmente asociada a cada núcleo (puede ser compartida por pocos núcleos).
- **L3:** suele ser compartida entre varios núcleos.

SIMULACIÓN

En el siguiente enlace podéis encontrar una simulación interactiva de la Arquitectura de Von Neumann:

https://lab.xitrus.es/VonNeumann/

el siguiente enlace podéis encontrar una simulación interactiva de la Arquitectura de Von Neumann.

Quiero que ejecutéis al menos un ciclo explicando en vuestras propias palabras lo que está sucediendo en un documento.

CONJUNTO DE INSTRUCCIONES

La CPU dispone de un conjunto definido de instrucciones que puede ejecutar para realizar distintas tareas.

Estas instrucciones son comandos codificados que le indican a la CPU qué operaciones realizar.

Las instrucciones se suelen clasificar en tres grandes grupos según su función principal:

- **Operaciones aritmético-lógicas:** Son aquellas que permiten realizar cálculos matemáticos (como suma, resta, multiplicación, división) y operaciones lógicas (como AND, OR, XOR, NOT) sobre datos almacenados en registros o memoria.

CONJUNTO DE INSTRUCCIONES (II)

Grupos de instrucciones según su función principal (II):

- **Operaciones de manejo de datos y memoria:** Permiten transferir datos entre la CPU y la memoria principal o dispositivos de entrada/salida. Incluyen instrucciones para leer (cargar) datos de la memoria o dispositivos, así como para almacenar (guardar) resultados en memoria o enviar datos a dispositivos externos.

- **Operaciones de control de flujo:** Modifican el orden secuencial en que se ejecutan las instrucciones del programa. Esto se logra cambiando el valor del **Contador de Programa** (Program Counter, PC), lo que permite implementar saltos condicionales o incondicionales, llamadas a funciones, retornos, y bucles.

CICLO DE INSTRUCCIÓN EN UNA CPU

El **ciclo de instrucción** es el proceso básico mediante el cual la CPU obtiene, interpreta y ejecuta cada instrucción del programa.

Este ciclo se repite continuamente mientras la computadora está encendida y ejecutando código.

Cada instrucción en la CPU está compuesta por un conjunto de bits.

La longitud típica de una instrucción está ligada al **tamaño de palabra** de la CPU (por ejemplo, 16, 32 o 64 bits), que determina cuántos bits puede procesar o manejar la CPU en una operación.

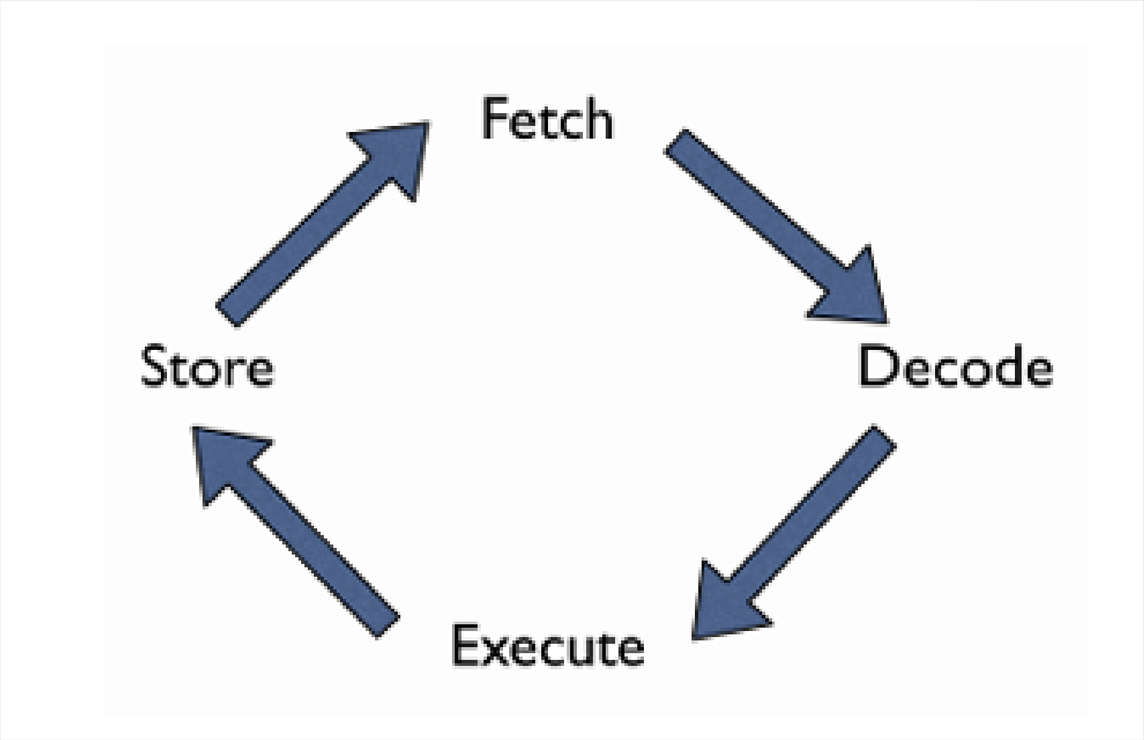
CICLO DE INSTRUCCIÓN EN UNA CPU (II)

Una instrucción generalmente consta de dos campos principales:

- **Código de operación (Opcode):** Es el código que indica a la CPU qué tipo de operación debe realizar, por ejemplo, sumar, cargar un dato, saltar a otra instrucción, etc.

- **Operandos:** Son los datos específicos o direcciones sobre los cuales se realiza la operación. Estos pueden ser valores inmediatos, registros o direcciones de memoria.

FASES DETALLADAS DEL CICLO DE INSTRUCCIÓN



1. BÚSQUEDA (FETCH)

- La CPU utiliza la dirección almacenada en el **Contador de Programa (PC)** para acceder a la siguiente instrucción en la memoria principal.

- La instrucción completa se carga en el **Registro de Instrucción (RI)**.

- El PC se incrementa automáticamente para apuntar a la dirección de la siguiente instrucción, preparándose para la siguiente iteración del ciclo.

2. DECODIFICACIÓN (DECODE)

- La unidad de control interpreta el código de operación (opcode) almacenado en el RI (registro de instrucción).

- Se determina qué tipo de instrucción es y qué componentes de la CPU estarán involucrados en su ejecución.

- Se identifican los operandos necesarios y su ubicación (registros, memoria, dispositivos).

3. EJECUCIÓN (EXECUTE)

- La CPU realiza la operación solicitada.

- Por ejemplo, puede tomar datos de registros o memoria, pasarlos a la Unidad Aritmético-Lógica (ALU) para efectuar cálculos o comparaciones, o realizar transferencias de datos.

- La unidad de control emite las señales adecuadas para coordinar estos procesos.

4. ALMACENAMIENTO (STORE)

- El resultado de la operación se escribe en el destino indicado, que puede ser un registro interno de la CPU, una dirección de memoria o un dispositivo externo.

- Si la instrucción implica modificar el flujo de control (como saltos), el PC puede actualizarse con una dirección diferente para continuar la ejecución desde otro punto del programa.

IMPORTANCIA DEL CICLO DE INSTRUCCIÓN

- Este ciclo básico es el motor que permite que la CPU ejecute cualquier programa, desde sistemas operativos hasta aplicaciones de usuario.

- El diseño eficiente del ciclo de instrucción es crucial para el rendimiento de la CPU.

- En arquitecturas modernas, existen **técnicas para acelerar este ciclo**, como ***pipelines***, **ejecución fuera de orden** y **predicción de saltos**, que permiten procesar múltiples instrucciones simultáneamente o anticipar pasos futuros.

EJEMPLO DIDÁCTICO DE UN CONJUNTO DE INSTRUCCIONES

A continuación, describimos un conjunto de instrucciones ficticio de un ordenador imaginario para entender qué es un conjunto de instrucciones y cómo se codifica una instrucción.

- Cada instrucción está codificada usando 8 bits.
- Los primeros 2 bits indican la operación a realizar.
- Los siguientes 6 bits almacenan información relacionada con la operación (como direcciones de memoria).
- La memoria tiene 26=642^6 = 6426=64 direcciones, cada una con una palabra de 8 bits (1 byte).
- La CPU tiene 3 registros temporales: R1, R2 y R3, que almacenan 8 bits cada uno.
- En la memoria:
  - La dirección 000000 contiene la palabra 00001001 (9 en complemento a dos).
  - La dirección 000001 contiene la palabra 00001000 (8 en complemento a dos).

CONJUNTO DE INSTRUCCIONES

Código binario	Operación
00	Sumar el contenido de los registros R1 y R2, almacenar el resultado en R3. Los siguientes 6 bits se ignoran.
01	Cargar en R1 el contenido de la dirección de memoria indicada en los 6 bits siguientes.
10	Cargar en R2 el contenido de la dirección de memoria indicada en los 6 bits siguientes.
11	Mostrar en pantalla el contenido del registro R3. Los siguientes 6 bits se ignoran.

EJEMPLO DE CODIFICACIÓN DE UNA INSTRUCCIÓN

- Si queremos cargar en R2 el valor almacenado en la dirección 000010, la instrucción se codifica así: 10 000010
  - 10 indica la operación de carga en R2.
  - 000010 es la dirección de memoria que queremos leer.

PROGRAMA SENCILLO

El siguiente código realiza la suma de los números 9 y 8 almacenados en memoria y muestra el resultado en pantalla.

Instrucción codificada	Descripción
01 000000	Cargar en R1 el contenido de la dirección 000000 (valor 9).
10 000001	Cargar en R2 el contenido de la dirección 000001 (valor 8).
00 000000	Sumar R1 y R2, almacenar el resultado en R3.
11 000000	Mostrar en pantalla el valor de R3 (resultado de la suma).

EJECUCIÓN PASO A PASO DEL PROGRAMA

Tratad de pensar cuál sería la ejecución paso a paso de este programa...

SOLUCIÓN

Estado inicial:

- Registros R1, R2 y R3 vacíos.
- Memoria con:
  - Dirección 000000 = 9
  - Dirección 000001 = 8

Paso 1: 01 000000 — Cargar en R1 el valor en la dirección 000000

- La CPU interpreta la instrucción.
- Lee el valor 9 en la dirección 000000.
- Guarda 9 en el registro R1.

Estado:

R1 = 9, R2 = vacío, R3 = vacío.

SOLUCIÓN (II)

Paso 2: 10 000001 — Cargar en R2 el valor en la dirección 000001

- La CPU interpreta la instrucción.
- Lee el valor 8 en la dirección 000001.
- Guarda 8 en el registro R2.

Estado:

R1 = 9, R2 = 8, R3 = vacío.

Paso 3: 00 000000 — Sumar R1 y R2, almacenar en R3

- La CPU interpreta la instrucción.
- Suma 9 (R1) + 8 (R2) = 17.
- Guarda 17 en el registro R3.

Estado:

R1 = 9, R2 = 8, R3 = 17.

Paso 4: 11 000000 — Mostrar en pantalla el valor de R3

- La CPU interpreta la instrucción.
- Envía el valor 17 de R3 al dispositivo de salida para mostrarlo en pantalla.

RESUMEN VISUAL DEL PROGRAMA

Instrucción	Acción	Registro afectado	Valor almacenado
01 000000	Cargar en R1 desde memoria dirección 000000	R1	9
10 000001	Cargar en R2 desde memoria dirección 000001	R2	8
00 000000	Sumar R1 + R2 y guardar en R3	R3	17
11 000000	Mostrar valor de R3 en pantalla	—	17

ACTIVIDADES NO EVALUABLES 01
EJERCICIO 03 SOLUCIÓN (II)

Code	Instruction	Description
ENT M(m)	**000**mmmmm	Leer datos del teclado a memoria.
SAL M(m)	**001**mmmmm	Mostrar datos en pantalla desde memoria.
CAR R0, M(m)	**010**mmmmm	Cargar el contenido de una dirección de memoria en el registro R0.
ALM M(m), R0	**011**mmmmm	Almacenar el contenido de R0 en una dirección de memoria.
MOV Rx, Ry	**1000**xxyy	Copiar el contenido de RY en RX (X, Y son números de registro).
SUM Rx, Ry	**1001**xxyy	Sumar RX + RY, y el resultado se almacena en RX.
RES Rx, Ry	**1010**xxyy	Restar RX – RY, y el resultado se almacena en RX.
MUL Rx, Ry	**1011**xxyy	Multiplicar RX * RY, y el resultado se almacena en RX.
DIV Rx, Ry	**1100**xxyy	Dividir RX / RY, y el resultado se almacena en RX.

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN

00001011 (A) 00001100 (B) 00010001 (C) 00011100 (D)
01001011 10000100 01011100 10001100
01010001 10001000 10111110 10101101
01001100 10001000 10011110 01010001
10001000 11001110 10000011 01101101 00101101

Donde A, B, C y D representan los datos introducidos mediante el teclado y sus valores son:

- A = 1
- B = 2
- C = 3
- D = 4

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN

- **00001011** Escribir el valor obtenido del teclado (A) en la posición de memoria 11 (A)
  Escribir 1 (obtenido del teclado)
- **00001100** Escribir el valor obtenido del teclado (B) en la posición de memoria 12 (B)
  Escribir 2 (obtenido del teclado)
- **00010001** Escribir el valor obtenido del teclado (C) en la posición de memoria 17 (C)
  Escribir 3 (obtenido del teclado)
- **00011100** Escribir el valor obtenido del teclado (D) en la posición de memoria 28 (D)
  Escribir 4 (obtenido del teclado)

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN (II)

- **01001011** Copiar los datos de la posición de memoria 11 al registro 0
- **10000100** Copiar los datos del registro 0 al registro 1: 1 → en R1

- **01011100** Copiar los datos de la posición de memoria 28 al registro 0
- **10001100** Copiar los datos del registro 0 al registro 3: 4 → en R3

- **01010001** Copiar los datos de la posición de memoria 17 al registro 0
- **10001000** Copiar los datos del registro 0 al registro 2: 3 → en R2

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN (III)

- **10111110** Multiplicar el contenido de R3 y R2 y escribir el resultado en R3 [3*4] → 12 en R3
- **10101101** Restar el contenido de R3 y R1 y escribir el resultado en R3 [12–1] → 11 en R3

- **01001100** Copiar los datos de la posición de memoria 12 al registro 0
- **10001000** Copiar los datos del registro 0 al registro 2: 2 → en R2

- **10011110** Sumar el contenido de R3 y R2 y escribir el resultado en R3 [2 + 11] → 13 en R3
- **01010001** Copiar los datos de la posición de memoria 17 al registro 0

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN (IV)

- **10001000** Copiar los datos del registro 0 al registro 2: 3 → en R2
- **11001110** Dividir el contenido de R3 entre R2 y escribirlo en R3 [13/3] → 4 en R3

- **10000011** Copiar los datos de R3 a R0 → 4 en R0
- **01101101** Escribir en la posición de memoria 13 el contenido del registro 0

- **00101101** Mostrar en la pantalla el contenido de la posición de memoria 13

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN (V)

- **Fórmula:** ((D*C) – A + B) / C
- **Resultado mostrado en pantalla:** 4 (contenido de la posición de memoria 13).
- **Estado de memoria y registros:** el mostrado en la siguiente transparencia.
- **Contador de Programa (PC):** si inicialmente estaba en 258 y se han ejecutado 21 instrucciones, el PC tendrá el valor 279.
- **Registros de propósito general:** tenemos dos bits, es decir, 4 registros.

ACTIVIDADES NO EVALUABLES 01 EJERCICIO 03 SOLUCIÓN A (VI)

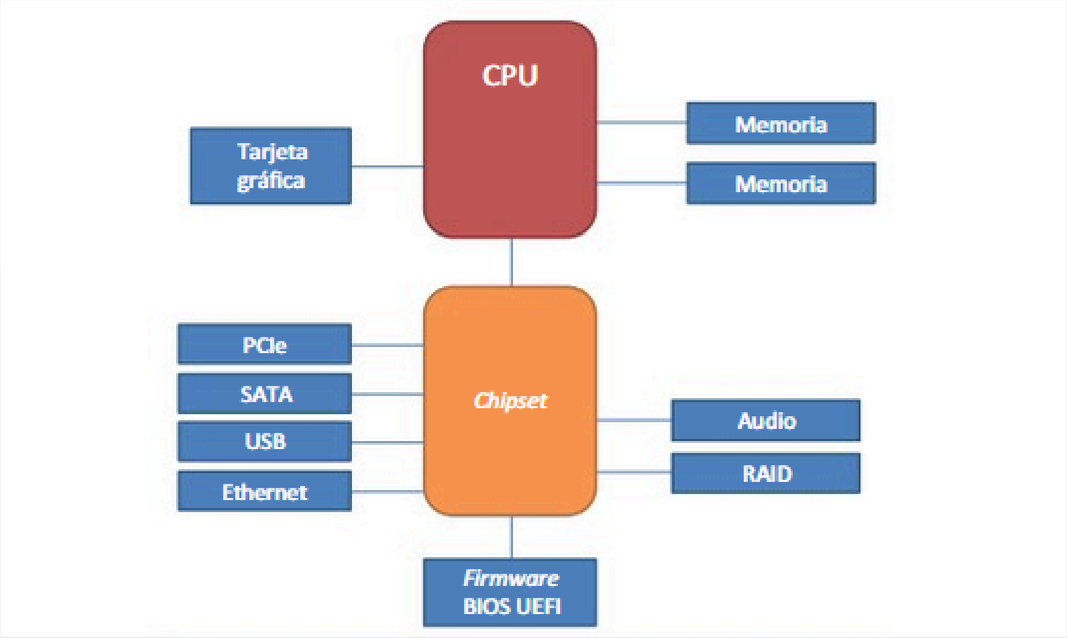
- **Estado de memoria y registros**

Memory																	
0									16								
1									17	0	0	0	0	0	0	1	1
2									18								
3									19								
4									20								
5									21								
6									22								
7									23								
8									24								
9									25								
10									26								
11	0	0	0	0	0	0	0	1	27								
12	0	0	0	0	0	0	1	0	28	0	0	0	0	0	1	0	0
13	0	0	0	0	0	1	0	0	29								
14									30								
15									31								

Registers								
R0	0	0	0	0	0	1	0	0
R1	0	0	0	0	0	0	0	1
R2	0	0	0	0	0	0	1	1
R3	0	0	0	0	0	1	0	0

HARDWARE DE UN SISTEMA INFORMÁTICO

Anteriormente se vieron los elementos funcionales del ordenador (realizan una función concreta en su interior). Es el momento de ver los elementos físicos, también llamados componentes *hardware*. El esquema físico de un ordenador sería:



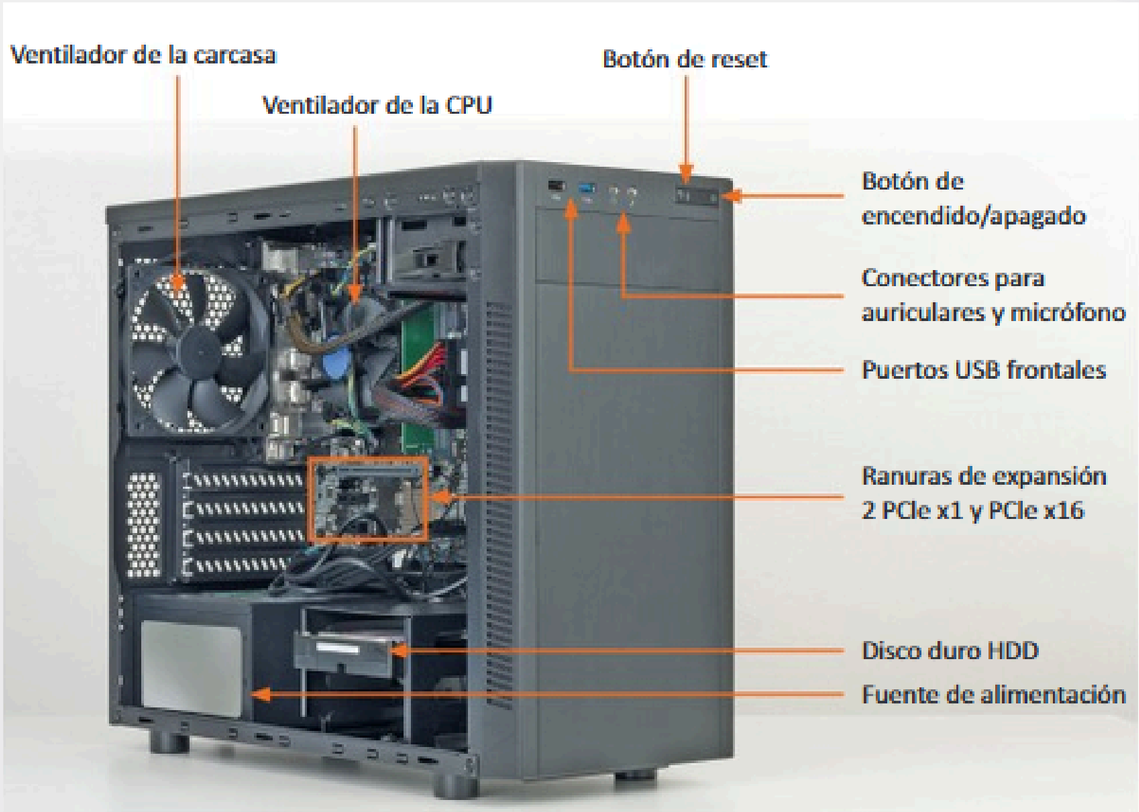
HARDWARE DE UN SISTEMA INFORMÁTICO (II)

El ***chipset*** o conjunto de chips se encarga de las comunicaciones entre los distintos componentes de la placa base con el microprocesador.

La placa base incluía los *chipsets* denominados ***northbridge*** (puente norte) y ***southbridge*** (puente sur), pero las más recientes integran el puente norte dentro del chip del microprocesador, para evitar colapsos en los buses de la placa base, ya que el puente norte tiene asociadas la memoria RAM y la tarjeta gráfica, que son las que necesitan mayor velocidad y mayor cantidad de datos.

CARCASA O CAJA DEL ORDENADOR

Puede ser de un ordenador de sobremesa, un portátil o cualquier otro dispositivo con arquitectura de ordenador. Dependiendo de este existen diferentes tipos.



TIPOS DE CARCASAS

Existen diferentes tipos de carcasas o chasis que se diferencian por su tamaño o por el factor de forma.

Este factor depende directamente de la placa base y de la fuente de alimentación. Es decir, el tamaño de la carcasa incidirá en el tamaño de la placa base, y viceversa.

Los principales tipos de carcasas para ordenadores son las siguientes de mayor a menor tamaño: torre, semitorre, minitorre, de sobremesa o *desktop*, *slim* o formato pequeño, *barebone*, *all in one*... entre otros.

FUENTE DE ALIMENTACIÓN

- Transforma la corriente alterna de la red eléctica en corriente continua de varios voltajes que es la que necesitan los ordenadores para funcionar.

- **Ordenador de sobremesa:** la fuente de alimentación será interna y puede ser de varios tipos dependiendo del tamaño de la fuente (ATX, SFX, TFX, dependiendo de la carcasa donde se ubique y de la placa base que se vaya a alimentar.

- **Pequeños ordenadores y portátiles:** la fuente puede ser externa. En los portátiles además alimentará una batería para que el equipo tenga un tiempo de autonomía sin necesidad de conectarse a la red eléctrica

FUENTE DE ALIMENTACIÓN (II)

Una de las mejoras de las fuentes de alimentación internas son las **fuentes de alimentación modulares.** Se caracterizan porque no tienen los cables inicialmente conectados, sino que se van añadiendo a medida que se van necesitando. La ventaja que aportan es la comodidad de no tener cables dentro de la carcasa que no se van a utilizar, además de evitar cortos o derivaciones no deseadas.

Figura 1.39. *Fuente de alimentación interna modular para un equipo de sobremesa. Los cables se irán conectando solo si se necesitan.*

FUENTE DE ALIMENTACIÓN (III)

Dependiendo de la carcasa y de la placa base se debe utilizar un tipo de fuente de alimentación u otro; por tanto, habrá que elegir una fuente de alimentación que se adapte tanto en potencia como en factor de forma a la placa base y a la carcasa.

La cantidad de vatios que tenga una fuente de alimentación indica su potencia, lo que se traduce en que podrá tener más dispositivos internos y dispositivos que consuman más.

Si a una fuente con poca potencia se le conectan demasiados dispositivos puede provocar que el sistema no arranque o que no funcione adecuadamente.

FUENTE DE ALIMENTACIÓN (IV)

La fuente de alimentación dispone de una serie de conectores que alimentará la placa base y otros dispositivos:

- **Conector de alimentación ATX de 24 pines:** para conectar la placa base.
- **Conector del procesador ATX de 4 pines o EPS de 8 pines:** para conectar el procesador.
- **Conector de alimentación *molex* de 4 pines:** en desuso, se utilizaba para conectar los discos duros y las unidades ópticas de tipo IDE.
- **Conector para tarjeta gráfica de 6 u 8 pines.**
- **Conector de alimentación SATA:** ha sustituido al conector *molex* y proporciona energía a los dispositivos.

¿PREGUNTAS?